



SAPIENZA
UNIVERSITÀ DI ROMA

Metodi per la progettazione di sistemi intelligenti

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea in Informatica

Candidato

András Boguslaw Arancio

Matricola 1941005

A stylized, handwritten signature in black ink, likely belonging to András Boguslaw Arancio.

Responsabile del tirocinio

Prof. Enrico Tronci

A handwritten signature in purple ink, reading "Enrico Tronci".

Anno Accademico 2023/2024

Tesi non ancora discussa

Metodi per la progettazione di sistemi intelligenti

Relazione di tirocinio. Sapienza Università di Roma

© 2024 András Boguslaw Arancio. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: arancio.1941005@studenti.uniroma1.it

Sommario

Questo lavoro si propone di formulare e analizzare una soluzione algoritmica a problemi di controllo complessi, prima concentrandosi sul problema relativo al Cart-Pole environment della libreria Python Gymnasium¹, di cui si utilizza una versione modificata, poi estendendo l'analisi ad una varietà di casi più ampia. La ricerca si articola in una progressione di soluzioni, inizialmente basate sulla Dynamic Programming (nello specifico si è implementato l'algoritmo Q-learning²), successivamente su tecniche moderne che impiegano Neural Networks per l'approssimazione delle cost-functions.

I risultati ottenuti mostrano che, nonostante la Dynamic Programming garantisca la convergenza alla cost-function ottimale, i costi computazionali elevati ne limitano l'applicabilità. L'uso delle Neural Networks ha permesso di migliorare i tempi di calcolo e operare in spazi di stato continui, ma con prestazioni inizialmente instabili. Introducendo miglioramenti all'algoritmo Deep Q Network (DQN)³, nello specifico l'Experience Replay⁴, il multi-step bootstrapping⁵ e il Double DQN⁶, è stato possibile ottenere una pseudo-convergenza alla cost-function ottimale in tutte le applicazioni esaminate, dimostrando l'efficacia di tali approcci nel risolvere problemi di controllo complessi.

¹Gymnasium, An API standard for reinforcement learning with a diverse collection of reference environments. <https://gymnasium.farama.org>.

²Watkins C.J.C.H. (1992), *Technical Note, Q-learning*. Machine Learning, 8:279-292.

³Mnih V., Kavukcuoglu K., Silver D., Rusu A.A., Veness J., Bellemare M.G., Graves A., Riedmiller M., Fidjeland A.K., Ostrovski G., Petersen S., Beattie C., Sadik A., Antonoglou I., King H., Kumaran D., Wierstra D., Legg S., Hassabis D. (2015), *Human-level control through deep reinforcement learning*. Nature, 518:529-533. e Mnih V., Kavukcuoglu K., Silver D., Graves A., Antonoglou I., Wierstra D., Riedmiller M. (2023), *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602.

⁴Lin L. (1992), *Self-Improving Reactive Agents Based On Reinforcement Learning, Planning and Teaching*. Machine Learning, 8:293-321.

⁵Hernandez-Garcia J.F., Sutton R.S. (2019), *Understanding Multi-Step Deep Reinforcement Learning: A Systematic Study of the DQN Target*. arXiv:1901.07510.

⁶van Hasselt H., Guez A., Silver D. (2015), *Deep Reinforcement Learning with Double Q-learning*. arXiv:1509.06461.

Indice

1	Introduzione	1
2	Dynamic Programming	2
2.1	Problemi ad orizzonte finito deterministici	2
2.1.1	Problemi ad orizzonte finito, caso discreto	2
2.2	Q-Factors	4
2.3	Problemi ad orizzonte finito stocastici	5
2.4	Problemi stocastici stazionari ad orizzonte infinito	6
2.4.1	Metodologie per problemi ad orizzonte infinito	6
2.5	Q-Learning	10
3	Introduzione al CartPole Environment e formulazione del problema di Dynamic Programming associato	13
3.1	Descrizione del sistema	14
3.2	Definizione del problema di DP associato	15
3.2.1	Dinamica delle transizioni e dei guadagni	15
3.3	Discretizzazione del sistema	15
3.4	Limitazioni nell'osservabilità e ricostruzione delle variabili di stato	19
4	Architetture parametriche e Neural Networks	21
4.1	Architetture parametriche di approssimazione	21
4.1.1	Architetture Feature-Based	21
4.1.2	Training di architetture lineari e non lineari	23
4.2	Neural Networks	25
4.2.1	Single Layer Perceptron	26
4.2.2	Training delle Neural Networks	27
4.2.3	Backpropagation	28
4.3	Deep Q Network	31
5	Utilizzo di DQN per CartPole e altri problemi di controllo	33
5.1	Problemi riscontrati e soluzioni adottate	33
5.1.1	Deadly Triad	34
5.1.2	Migliorare l'algoritmo DQN	37
5.2	Soluzione del problema CartPole ad osservabilità ridotta	40
5.3	Utilizzo di DQN per altri problemi di controllo e analisi dei risultati	41
6	Conclusioni	49
	Bibliografia	50

Capitolo 1

Introduzione

La ricerca in oggetto nasce con l'intenzione di formulare una strategia risolutiva ad un problema di controllo. Nello specifico, ci proporremo di risolvere una versione modificata del problema relativo al *CartPole environment* della libreria Python *Gymnasium*¹, in cui l'obiettivo è "insegnare" ad un controllore (o agente), tramite tecniche di *Reinforcement Learning*, a mantenere in equilibrio un'asta attaccata ad un carrello mobile controllabile, partendo da qualunque posizione iniziale.

Per far ciò vedremo come risolvere il problema elaborando soluzioni gradualmente più complesse. Utilizzeremo inizialmente tecniche di Reinforcement Learning "classico", in grado di dare soluzioni esatte, ma con importanti limitazioni dovute alla *Curse of Dimensionality*. Successivamente saranno utilizzate tecniche moderne per l'approssimazione delle cost-functions, prendendo spunto da paper di più recente pubblicazione². I capitoli 2 e 4 trattano rispettivamente della teoria alla base dell'algoritmo di Dynamic Programming utilizzato per dare una soluzione al problema CartPole e della teoria alla base delle architetture parametriche, in particolare delle *Neural Networks*; per entrambi i capitoli, le argomentazioni sono principalmente tratte da *Reinforcement Learning and Optimal Control* (Bertsekas, 2019) e *A Course in Reinforcement Learning* (Bertsekas, 2023). I capitoli 3 e 5 presentano una traduzione di tale teoria in implementazione: si presentano gli algoritmi *Q-learning*³ e *Deep Q Network*⁴ e si analizzano i risultati ottenuti. Ancora, nel capitolo 5 si introducono delle migliorie all'algoritmo Deep Q Network⁵, per migliorarne la stabilità e le performance; inoltre, si applica l'algoritmo, in tutte le sue varianti, ad altri problemi di controllo e si analizzano i risultati ottenuti.

¹Gymnasium, An API standard for reinforcement learning with a diverse collection of reference environments. <https://gymnasium.farama.org>.

²In particolare, Mnih V., Kavukcuoglu K., Silver D., Rusu A.A., Veness J., Bellemare M.G., Graves A., Riedmiller M., Fidjeland A.K., Ostrovski G., Petersen S., Beattie C., Sadik A., Antonoglou I., King H., Kumaran D., Wierstra D., Legg S., Hassabis D. (2015), *Human-level control through deep reinforcement learning*. Nature, 518:529-533. e Mnih V., Kavukcuoglu K., Silver D., Graves A., Antonoglou I., Wierstra D., Riedmiller M. (2023), *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602.

³Watkins C.J.C.H. (1992), *Technical Note, Q-learning*. Machine Learning, 8:279-292.

⁴Gli stessi della nota 2.

⁵Tratte da Lin L. (1992), *Self-Improving Reactive Agents Based On Reinforcement Learning, Planning and Teaching*. Machine Learning, 8:293-321., Hernandez-Garcia J.F., Sutton R.S. (2019), *Understanding Multi-Step Deep Reinforcement Learning: A Systematic Study of the DQN Target*. arXiv:1901.07510. e van Hasselt H., Guez A., Silver D. (2015), *Deep Reinforcement Learning with Double Q-learning*. arXiv:1509.06461.

Capitolo 2

Dynamic Programming¹

2.1 Problemi ad orizzonte finito deterministici

Nel contesto della Programmazione Dinamica (DP) un problema ad orizzonte finito è un problema che ha per oggetto un sistema la cui evoluzione avviene in un numero finito di passi (anche chiamati stages). Lo stato del sistema ed il controllo applicato allo stage k si denotano rispettivamente x_k e u_k . Nei sistemi deterministici, x_{k+1} è generato in maniera causale, ossia è univocamente determinato da x_k ed u_k . Un problema di DP deterministico ha dunque per oggetto un sistema della forma

$$x_{k+1} = f_k(x_k, u_k), \quad k = 0, 1, \dots, N-1. \quad (2.1)$$

Il costo della transizione allo stage k è denotata come $g_k(x_k, u_k)$. Accumulando tali costi nel tempo otteniamo la cost-function:

$$J(x_0; u_0, \dots, u_{N-1}) = g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k). \quad (2.2)$$

Partendo dallo stato iniziale x_0 , utilizziamo la sequenza di controlli $\{u_0, \dots, u_{N-1}\}$ che minimizza la 2.2 per ottenere la cost function ottimale:

$$J^*(x_0) = \min_{u_k \in U(x_k)} J(x_0; u_0, \dots, u_{N-1}), \quad (2.3)$$

dove $u_k \in U(x_k)$ indica che il controllo u_k è nello spazio dei controlli dello stato x_k .

Nota: Per alleggerire la notazione, nel seguito ometterò la dichiarazione d'appartenenza $u_k \in U(x_k)$, che verrà assunta come implicita, e scriverò semplicemente u_k . La sequenza di controlli $\{u_0^*, \dots, u_{N-1}^*\}$ che minimizza l'equazione 2.3 è detta ottimale.

2.1.1 Problemi ad orizzonte finito, caso discreto

Ci restringiamo adesso alla classe di problemi in cui lo spazio degli stati e dei controlli assumono dimensioni finite. Questo tipo di problemi sono spesso ben descritti da grafi diretti aciclici, in cui ogni nodo rappresenta uno stato x_k e gli archi rappresentano la transizione corrispondente alla coppia stato-comando (x_k, u_k) (cfr. figura 2.1).

¹Il presente capitolo segue fedelmente nella trattazione degli argomenti, inclusi teoremi, dimostrazioni, esempi e algoritmi, i testi Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific e Bertsekas D.P. (2019), *Reinforcement Learning and Optimal Control*, Belmont, Massachusetts, Athena Scientific. Le eccezioni sono documentate.

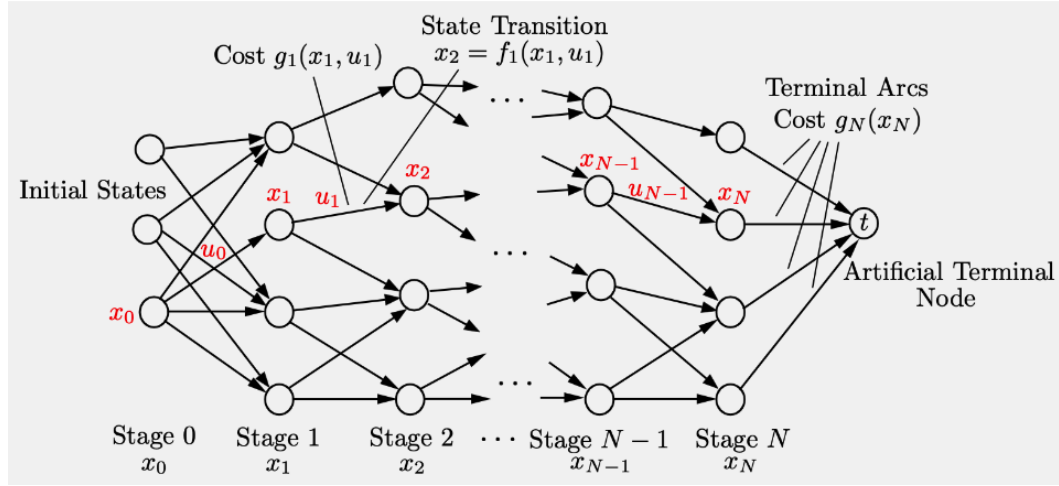


Figura 2.1. Rappresentazione tramite grafo di un problema deterministico ad orizzonte finito con spazio di stato e spazio di controlli a dimensione finita. (Fonte: Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific, p.11)

Theorem 1 (Principle of Optimality (Bertsekas, 2023, p.13)). *Sia $\{u_0^*, \dots, u_{N-1}^*\}$ una sequenza di controlli ottimale che assieme allo stato iniziale x_0 determina la sequenza di stati $\{x_1^*, \dots, x_N^*\}$. Consideriamo il sotto-problema che ha per stato iniziale x_k^* ed inizia al tempo k . Cerchiamo la sequenza $\{u_1, \dots, u_k\}$ che minimizza la seguente somma:*

$$g_k(x_k^*, u_k) + \sum_{m=k+1}^{N-1} g_m(x_m, u_m) + g_N(x_N)$$

Si dimostra facilmente che la sequenza troncata $\{u_k^, \dots, u_{N-1}^*\}$ è una soluzione ottimale per tale sotto-problema.*

Il sotto-problema citato nel teorema 1 è spesso chiamato tail-subproblem. Basandosi sulla proprietà per cui la coda della sequenza ottimale è soluzione del tail-subproblem corrispondente, si formula l'algoritmo di DP (Teorema 2). Tale algoritmo procede sequenzialmente, calcolando le soluzioni per i tail-subproblems con istante iniziale k utilizzando le soluzioni dei tail-subproblems con istante iniziale $k+1$.

Theorem 2 (Algoritmo di DP per problemi deterministici ad orizzonte finito (Bertsekas, 2023, p.17)). *Si inizia assegnando il costo dello stato finale*

$$J_N^*(x_N) = g_N(x_N), \quad \text{per ogni } x_N,$$

e per ogni $k = 0, \dots, N-1$ si assegna

$$J_k^*(x_k) = \min_{u_k} \left[g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k)) \right], \quad \text{per ogni } x_k. \quad (2.4)$$

Il costo $J_0^*(x_0)$ ottenuto utilizzando l'algoritmo di DP è esattamente il costo ottimo $J^*(x_0)$, per ogni stato iniziale x_0 . Più in generale, $J_k^*(x_k)$ è il costo ottimale del tail sub-problem con stato iniziale x_k e istante iniziale k , per ogni k, x_k .

Una volta note le funzioni $J_0^*, \dots, J_N^*$, possiamo utilizzare un algoritmo che procedendo in avanti costruisce una sequenza di controlli ottimale ed una corrispondente "traiettoria" di stati, per un dato stato iniziale x_0 .

Theorem 3 (Algoritmo per la costruzione della sequenza ottimale di controlli (Bertsekas, 2023, p.19)). *Sia*

$$u_0 \in \arg \min_{u_0} \left[g_0(x_0, u_0) + J_1^*(f_0(x_0, u_0)) \right],$$

e sia

$$x_1^* = f_0(x_0, u_0^*).$$

Procedendo sequenzialmente in avanti, per $k = 1, 2, \dots, N - 1$, si assegna

$$u_k^* \in \arg \min_{u_k} \left[g_k(x_k^*, u_k) + J_{k+1}^*(f_k(x_k^*, u_k)) \right],$$

e

$$x_{k+1}^* = f_k(x_k^*, u_k^*).$$

2.2 Q-Factors

Una forma alternativa ma equivalente dell'algoritmo di DP utilizza le funzioni di costo ottimale J_k^* indirettamente. In particolare, vengono generati i **Q-Factors ottimali**, definiti per ogni coppia (x_k, u_k) e k come

$$Q_k^*(x_k, u_k) = g_k(x_k, u_k) + J_{k+1}^*(f_k(x_k, u_k)). \quad (2.5)$$

Dunque, i Q-Factors ottimali non sono altro che le espressioni che vengono minimizzate nella parte alla destra dell'uguale nell'equazione 2.4 dell'algoritmo di DP. Sussiste quindi la relazione

$$J_k^*(x_k) = \min_{u_k} Q_k^*(x_k, u_k). \quad (2.6)$$

Riscriviamo a questo punto l'algoritmo di DP, utilizzando i Q-Factors

Theorem 4 (Algoritmo DP con Q-Factors (Bertsekas, 2023)). *Si inizia assegnando i costi*

$$Q_{N-1}(x_{N-1}, u_{N-1}) = g_{N-1}(x_{N-1}, u_{N-1}) + g_N(x_N) \quad (2.7)$$

e per ogni $k = 0, \dots, N - 2$ si assegna

$$Q_k^*(x_k, u_k) = g_k(x_k, u_k) + \min_{u_{k+1}} Q_{k+1}^*(f_k(x_k, u_k), u_{k+1}) \quad (2.8)$$

Notiamo che conoscendo i Q-Factors ottimali, possiamo costruire la sequenza ottimale di controlli attraverso le minimizzazioni $u_k^* = \arg \min_{u_k} Q_k^*(x_k, u_k)$.

Approssimazioni nello spazio del costo e dei controlli Limitandoci a quanto visto finora, per il calcolo delle sequenze di controlli ottimali abbiamo bisogno di conoscere la cost-function ottimali J_k^* oppure i Q-Factors ottimali Q_k^* . Purtroppo, il calcolo di tali funzioni non è sempre semplice: se la dimensione dello spazio di stato e dello spazio dei controlli è significativa, il calcolo di tali funzioni potrebbe essere computazionalmente proibitivo. Per ovviare a questo problema si ricorre a delle approssimazioni delle cost-functions o dei Q-Factors. Introduciamo dunque le funzioni $\tilde{J}_k$, $\tilde{Q}_k$ e la sequenza di controlli $\{\tilde{u}_1, \dots, \tilde{u}_{N-1}\}$ che le minimizza.

2.3 Problemi ad orizzonte finito stocastici

Esploriamo adesso il contesto più generale dei problemi stocastici. In questa sezione ci occupiamo della teoria relativa ai problemi ad orizzonte finito, e nella prossima sezione la estenderemo al caso stocastico ad orizzonte infinito. In quest'ultimo caso trova fondamento la teoria dietro agli algoritmi utilizzati per la soluzione dei problemi presentati nei capitoli successivi.

La differenza principale fra il caso deterministico e quello stocastico risiede nella natura del sistema a tempo discreto che governa l'evoluzione dello stato x_k . Nel caso stocastico, questo sistema include un disturbo randomico w_k con una propria distribuzione di probabilità $P_k(\cdot|x_k, u_k)$, che potrebbe dipendere da x_k, u_k ma non dai disturbi precedenti $w_{k-1}, \dots, w_0$. In questo modo, la funzione di transizione diventa

$$x_{k+1} = f_k(x_k, u_k, w_k), \quad k = 0, 1, \dots, N-1, \quad (2.9)$$

mentre il costo della transizione viene rappresentato come $g_k(x_k, u_k, w_k)$.

Dato uno stato iniziale x_0 ed una policy $\pi = \{\mu_0, \dots, \mu_{N-1}\}$ (dove ogni μ_k è una funzione che ad ogni x_k associa un comando $\mu_k(x_k) = u_k$), gli stati futuri e i disturbi w_k sono variabili aleatorie con distribuzioni definite dal sistema di equazioni

$$x_{k+1} = f_k(x_k, \mu_k(x_k), w_k), \quad k = 0, 1, \dots, N-1$$

e dalla distribuzione $P(\cdot|x_k, w_k)$ rispettivamente. Dunque, il costo atteso di una policy π è

$$J_\pi(x_0) = E_{w_k} \left\{ g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k), w_k) \right\}.$$

Una policy ottimale è tale da minimizzare il seguente costo

$$J_{\pi^*}(x_0) = \min_{\pi} J_\pi(x_0).$$

Va notato che la minimizzazione non avviene sulla sequenza dei comandi ma sulla policy. Il costo ottimo J^* è il costo della policy minimizzante J_{π^*} .

Quando nei capitoli successivi risolveremo il problema CartPole della libreria Gymnasium (<https://gymnasium.farama.org/>), utilizzeremo una metodologia consona a problemi stocastici; infatti, sebbene la dinamica del CartPole sia di fatto deterministica, saremo costretti a discretizzare lo spazio di stato, trasformando il problema in un nuovo problema stocastico e calcoleremo una policy che massimizzi il guadagno atteso.

Le policy sono oggetti più generali delle sequenze di controlli, queste permettono di scegliere il controllo u_k in base alla conoscenza che si ha dello stato x_k . Nel caso stocastico questo aspetto è fondamentale, ma è molto utile anche nei casi deterministici in cui non si conosce la funzione di transizione f . Al momento dell'implementazione del progetto approfondiremo questi concetti.

Adesso presentiamo l'algoritmo di DP per problemi stocastici ad orizzonte finito, che sarà fondamentale per le argomentazioni fatte nelle sezioni successive.

Theorem 5 (Algoritmo di DP per problemi stocastici ad orizzonte finito (Bertsekas, 2023, p.29)). *Si inizia assegnando il costo dello stato finale*

$$J_N^*(x_N) = g_N(x_N), \quad \text{per ogni } x_N,$$

e per ogni $k = 0, \dots, N - 1$ si assegna

$$J_k^*(x_k) = \min_{u_k} E_{w_k} \left\{ g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k)) \right\}, \quad \text{per ogni } x_k. \quad (2.10)$$

Per ogni x_k e k , si definisce $\mu_k^*(x_k) = u_k^*$ dove u_k^* è il comando che minimizza la parte destra dell'equazione. La policy $\pi^* = \{\mu_0^*, \dots, \mu_{N-1}^*\}$ è ottimale.

Q-Factors per problemi stocastici Come per il caso deterministico, possiamo definire i Q-Factors ottimali anche per i problemi stocastici. Questi sono dati da

$$Q_k^*(x_k, u_k) = E_{w_k} \left\{ g_k(x_k, u_k, w_k) + J_{k+1}^*(f_k(x_k, u_k, w_k)) \right\} \quad (2.11)$$

e l'algoritmo di DP può essere riscritto come

$$Q_k^*(x_k, u_k) = E_{w_k} \left\{ g_k(x_k, u_k, w_k) + \min_{u_{k+1}} Q_{k+1}^*(f_k(x_k, u_k, w_k), u_{k+1}) \right\}. \quad (2.12)$$

Come nel caso deterministico, si ricorre spesso ad una approssimazione della cost function $\tilde{J} \approx J^*$.

2.4 Problemi stocastici stazionari ad orizzonte infinito

In un problema ad orizzonte infinito, l'obiettivo è minimizzare la cost-function su un numero infinito di stages, il cui costo è dato da:

$$J_\pi(x_0) = \lim_{N \rightarrow \infty} \left\{ \sum_{k=0}^{N-1} \alpha^k g(x_k, \mu_k(x_k), w_k) \right\}. \quad (2.13)$$

In un problema stazionario, le funzioni f e g sono le stesse per ogni istante temporale, anche i disturbi $w_0, w_1, \dots$ condividono un'unica distribuzione di probabilità $P(\cdot|x_k, u_k)$. α è uno scalare in $(0, 1]$. Se $\alpha < 1$ allora viene chiamato *discount factor* e ha l'effetto di diminuire l'importanza dei costi futuri rispetto a quelli più prossimi. Inoltre, un discount factor garantisce l'esistenza del limite nella [2.13](#) (Bertsekas, 2023).

2.4.1 Metodologie per problemi ad orizzonte infinito

Ci concentriamo sui problemi con discount ($\alpha < 1$) per due motivi: 1) possiamo implementare il CartPole problem come un problema discounted, 2) per questo tipo di problemi esistono le dimostrazioni matematiche delle assunzioni necessarie a costruire una metodologia efficace.

Value Iteration

Theorem 6 (Value Iteration (Bertsekas, 2023, p.45)). *Sia $J_N(x)$ il costo ottimo del problema ad N stages con stato iniziale x . Questo costo è stato generato dopo N iterazioni dell'algoritmo Value Iteration*

$$J_{k+1}(x) = \min_u E_w \left\{ g(x, u, w) + J_k(f(x, u, w)) \right\}.$$

partendo da $J_0(x) \equiv 0$.

L'algoritmo di Value Iteration (VI), non è altro che l'algoritmo di DP per problemi stocastici ad orizzonte finito (Teorema 5) dove gli indici temporali sono stati invertiti per soddisfare le nostre esigenze in questo nuovo caso. Facciamo vedere la detta equivalenza.

Dimostrazione. ² Per far vedere l'equivalenza fra VI e DP stocastico, consideriamo il problema ad N stages e sia $V_{N-k}(x)$ il costo ottimale partendo da x , con k stages da affrontare, e costo terminale uguale a 0. Applicando il DP, abbiamo, per ogni x ,

$$V_{N-k}(x) = \min_u E_w \left\{ \alpha^{N-k} g(x, u, w) + V_{N-k+1}(f(x, u, w)) \right\}, \quad V_N(x) = 0.$$

Ponendo $J_k(x) = V_{N-k}(x)/\alpha^{N-k}$ otteniamo l'algoritmo 6. $\square$

I risultati fondamentali di nostro interesse su cui possiamo fare affidamento nel caso di problemi con discount, ossia dove si ha $\alpha < 1$ nella 2.13 sono i seguenti:

1. Il costo ottimale ad orizzonte finito è il limite per $N \rightarrow \infty$ del costo $J_N(x)$ ottenuto utilizzando l'algoritmo 6; cioè

$$J^*(x) = \lim_{N \rightarrow \infty} J_N(x) \quad (2.14)$$

per ogni stato x .

2. La seguente equazione, detta **equazione di Bellman** è verificata per ogni stato x

$$J^*(x) = \min_u E_w \left\{ g(x, u, w) + \alpha J^*(f(x, u, w)) \right\}. \quad (2.15)$$

Questa è ottenuta come il limite nell'algoritmo 6. È in verità un sistema di equazioni, una per ogni stato x .

3. Se $\mu(x)$ minimizza la parte destra dell'equazione di Bellmann 2.15 per ogni x , allora la policy $\mu = \{\mu, \mu, \dots\}$ è ottimale. Notiamo che π è una policy stazionaria, il che è ovvio se ci ricordiamo che g ed f sono le stesse per ogni stage k .
4. L cost function J_μ di una policy stazionaria μ soddisfa

$$J_\mu(x) = E_w \left\{ g(x, \mu(x), w) + \alpha J_\mu(f(x, \mu(x), w)) \right\}, \quad \forall x. \quad (2.16)$$

Possiamo pensare quest'ultima come l'equazione di Bellmann 2.15 per un problema diverso, dove per ogni x l'insieme dei controlli $U(x)$ per tale stato consiste di un unico controllo $\mu(x)$. Inoltre, il costo della policy J_μ è ottenuto come limite dell'algoritmo VI

$$J_\mu(x) = \lim_{N \rightarrow \infty} J_{\mu,N}(x), \quad \forall x,$$

²La dimostrazione è tratta da Bertsekas D.P. (2023), *A Course in Reinforcement Learning*. Belmont, Massachusetts, Athena Scientific, p. 45.

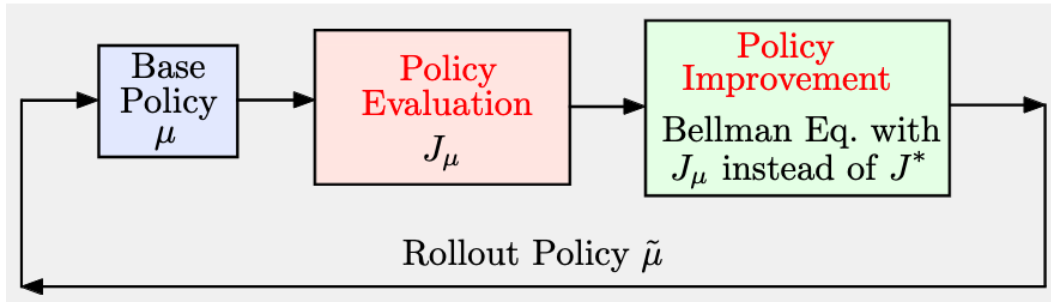


Figura 2.2. Il ciclo di Policy Iteration. (Fonte: Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific, p.47)

dove $J_{\mu,N}$ è la cost function della policy μ per il problema ad N stages

$$J_{\mu,k+1}(x) = E_w \left\{ g(x, \mu(x), w) + \alpha J_{\mu,k}(f(x, \mu(x), w)) \right\}, \quad (2.17)$$

partendo da $J_{\mu,0}(x) \equiv 0$ o una qualunque altra condizione iniziale.

Esistono diversi algoritmi di approssimazione di policy ottimali per problemi ad orizzonte infinito. Diamo un primo esempio per avere un'intuizione di come tali algoritmi funzionino, poi descriviamo l'algoritmo di Q-Learning (Watkins, 1992), utilizzato nella soluzione al problema affrontato nei capitoli successivi.

Policy Iteration

Un importante algoritmo per problemi ad orizzonte infinito è il *policy iteration*, PI (Bertsekas, 2019, ch.4:p.25). Questo algoritmo consiste di due fasi:

1. *Policy evaluation*, viene calcolato il costo J_μ della policy corrente μ . Una possibilità per calcolare tale costo è risolvere l'equazione di Bellman

$$J_\mu(x) = E_w \left\{ g(x, \mu(x), w) + \alpha J_\mu(f(x, \mu(x), w)) \right\}, \quad \forall x,$$

altrimenti è possibile utilizzare altri metodi come la simulazione MonteCarlo.

2. *Policy Improvement*, viene calcolata la nuova policy $\tilde{\mu}$ utilizzando la minimizzazione

$$\tilde{\mu}(x) \in \arg \min_u E_w \left\{ g(x, u, w) + \alpha J(f(x, u, w)) \right\}, \quad \forall x. \quad (2.18)$$

La condizione terminante è $J_{\mu^{k+1}}(x) = J_{\mu^k}(x) \quad \forall x$.

Proposition 1 (Convergenza dell'algoritmo di PI (Bertsekas, 2023)). *Per i problemi discounted, l'algoritmo di Policy Iteration produce una successione di policy sempre migliori*

$$J_{\mu^{k+1}}(x) \leq J_{\mu^k}(x) \quad \forall x, k, \quad (2.19)$$

e termina con una policy ottimale.

Dimostrazione. ³ Sia μ una policy e sia $\tilde{\mu}$ la policy ottenuta tramite la policy iteration, utilizzando μ come policy iniziale. Vogliamo far vedere che $J_{\tilde{\mu}} \leq J_{\mu}$. Denotiamo allora con J_N la cost-function di una policy che utilizza $\tilde{\mu}$ per i primi N stages e poi utilizza μ per ogni stato successivo all' N -esimo. Abbiamo l'equazione di Bellman

$$J_{\mu}(x) = E_w \left\{ g(x, \mu(x), w) + \alpha J_{\mu}(f(x, \mu(x), w)) \right\}$$

che assieme alla equazione di policy improvement 2.18 implica

$$J_1(x) = E_w \left\{ g(x, \tilde{\mu}(x), w) + \alpha J_{\mu}(f(x, \tilde{\mu}(x), w)) \right\} \leq J_{\mu}(x). \quad (2.20)$$

Dalle definizioni di J_2 e J_1 abbiamo che

$$J_2(x) = E_w \left\{ g(x, \tilde{\mu}(x), w) + \alpha J_1(f(x, \tilde{\mu}(x), w)) \right\}, \quad (2.21)$$

quindi le due relazioni precedenti implicano che

$$J_2(x) \leq J_1(x) \leq J_{\mu}(x), \quad \forall x. \quad (2.22)$$

Continuando secondo la stessa logica, otteniamo

$$J_{N+1}(x) \leq J_N(x) \leq J_{\mu}(x), \quad \forall x, N. \quad (2.23)$$

Dal momento che $J_N \rightarrow J_{\tilde{\mu}}$ (cfr. 2.15), segue che $J_{\tilde{\mu}} \leq J_{\mu}$. $\square$

Nella pratica, si ottiene un grande miglioramento del costo già dopo poche iterazioni dell'algoritmo.

Per gli algoritmi presentati a seguire, assumiamo che lo spazio di stato e dei controlli abbiano dimensione finita (in modo che il valore atteso si calcoli mediante una somma finita di termini).

Value Iteration con Q-Factors (discounted problems)

Adesso presentiamo l'algoritmo VI utilizzando i Q-Factors. VI con Q-Factors è un algoritmo versatile perchè una volta ottenuti i Q-Factors ottimali (o le loro approssimazioni) la policy μ può essere calcolata a runtime mediante la minimizzazione

$$\mu^*(x) = \arg \min_u Q^*(x, u), \quad \forall x.$$

Presentiamo l'equazione di Bellman nella sua forma con i Q-Factors. Per ogni stato x ed ogni controllo u si ha

$$Q^*(x, u) = E_w \left\{ g(x, u, w) + \alpha \min_u Q^*(f(x, u, w), u) \right\}. \quad (2.24)$$

A partire dall'equazione 2.24 definiamo l'algoritmo di VI per Q-Factors e problemi con discount.

³La dimostrazione è tratta da Bertsekas D.P. (2019), *Reinforcement Learning and Optimal Control*, Belmont, Massachusetts, Athena Scientific, ch.4:p.33.

Theorem 7 (VI con Q-Factors per discounted problems (Bertsekas, 2019)). *Per ogni stato x , controllo u e condizione iniziale $Q_0(x, u)$, l'algoritmo genera la successione $\{Q_k\}$ con*

$$Q_{k+1}(x, u) = E_w \left\{ g(x, u, w) + \alpha \min_u Q_k(f(x, u, w), u) \right\}. \quad (2.25)$$

Dato che questa versione del VI è equivalente alla 6, possiamo affermare che la successione generata dall'algoritmo 2.24 converge a $Q^*(x, u)$ per ogni x, u .

Policy Iteration per Q-Factors: problemi con discount

Theorem 8 (PI per Q-Factors, discounted (Bertsekas, 2019, ch.4:p.33)). *Data la policy della k -esima iterazione μ^k :*

1. Policy evaluation, *vengono calcolati $Q_{\mu^k}(x, u)$, per ogni stato x e ogni controllo u , risolvendo il sistema lineare*

$$Q_{\mu^k}(x, u) = E_w \left\{ g(x, u, w) + \alpha Q_{\mu^k}(f(x, u, w), \mu(f(x, u, w))) \right\}. \quad (2.26)$$

2. Policy improvement, *viene calcolata una nuova policy μ^{k+1}*

$$\mu^{k+1}(x) \in \arg \min_u Q_{\mu^k}(x, u) \quad \forall x. \quad (2.27)$$

La condizione terminante è $J_{\mu^{k+1}}(x) = J_{\mu^k}(x)$ per ogni x .

L'algoritmo PI per Q-Factors è matematicamente equivalente alla versione con le cost functions. L'unica differenza è che calcoliamo tutti i Q-Factors $Q_{\mu^k}(x, u)$, anzichè le cost functions $J_{\mu^k} = Q_{\mu^k}(x, \mu^k(x))$, ossia il Q-factor corrispondente al controllo ottimale. La cosa utile è che i restanti Q-factors $Q_{\mu^k}(x, u)$ vengono utilizzati al momento della policy improvement 2.27, per cui non è necessaria ulteriore computazione (necessaria nella versione con cost functions).

2.5 Q-Learning

L'algoritmo di Q-Learning (Watkins, 1992) per problemi con discount è un algoritmo off policy e model-free. Gli algoritmi model-free sono una tipologia di algoritmi che generano una policy senza utilizzare una conoscenza o una stima della distribuzione di probabilità associata alle transizioni di stato del sistema (Sutton & Barto, 2019).

Gli algoritmi off-policy imparano il costo della policy ottimale indipendentemente dalle azioni dell'agente (cioè indipendentemente dalla policy che sta utilizzando l'agente). Q-learning è un algoritmo off-policy perchè il Q-value di una certa coppia stato-azione viene aggiornato in base al Q-value dello stato successivo e dell'azione ottimale stimata per tale stato. Calcola dunque i Q-values assumendo di star usando una policy greedy, sebbene non stia effettivamente utilizzando una policy greedy.

Ancora, l'algoritmo di Q-learning si colloca nella categoria dei *Temporal Difference methods*, TD (Sutton & Barto, 2019). Il learning di questo tipo di algoritmi è basato su due comportamenti fondamentali. Da una parte, i metodi TD imparano

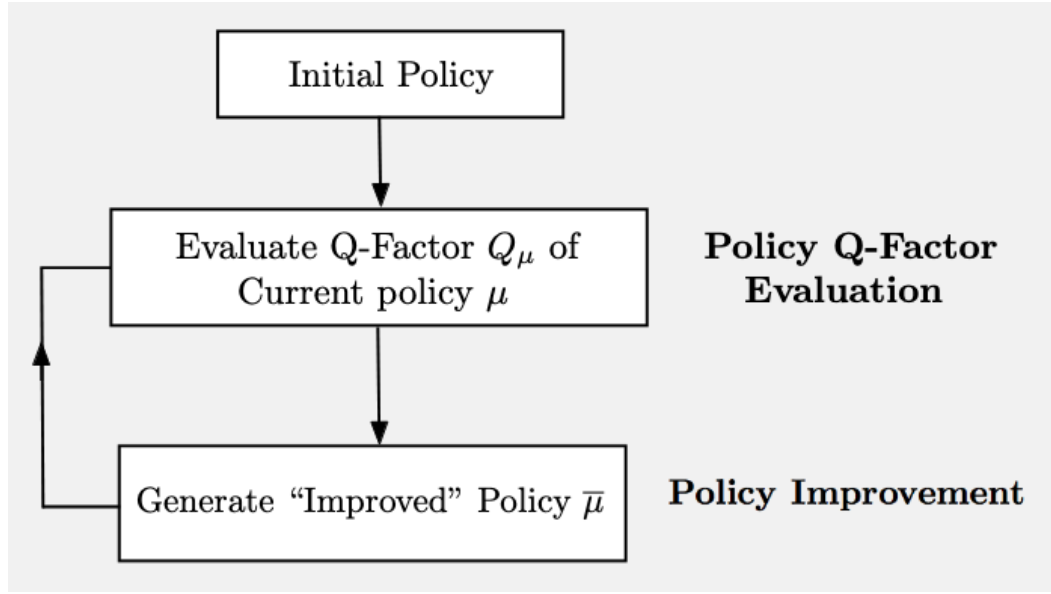


Figura 2.3. Policy Iteration per Q-Factors. (Fonte: Bertsekas D.P. (2019), *Reinforcement Learning and Optimal Control*, Belmont, Massachusetts, Athena Scientific, ch.4:p.33)

direttamente dall'esperienza, senza avere un modello della dinamica del sistema (sono dunque model-free). Inoltre, i metodi TD aggiornano le proprie stime dei costi a partire da altre stime imparate precedentemente. Questo meccanismo viene chiamato *bootstrap* (Sutton & Barto, 2019). È quello che succede nel Q-learning, quando si aggiorna il valore del Q-value a partire dal valore atteso del costo partendo dallo stato successivo.

L'algoritmo di Q-Learning originale deriva dall'algoritmo di VI 7. Anziché calcolare (o approssimare) il costo della policy successiva come nella fase di policy evaluation di PI 2.26, vengono aggiornati i Q-Factors associati ad una policy ottimale. In questo modo si evitano dispendiosi calcoli di cost evaluation.

Algoritmo di Q-Learning, versione discounted

Nei problemi discounted, i Q-Factors ottimali sono definiti, per ogni coppia (x, u) come

$$Q^*(x, u) = E_w \left\{ g(x, u, w) + \alpha J^*(f(x, u, w)) \right\}.$$

Una scrittura equivalente è

$$Q^*(x, u) = E_w \left\{ g(x, u, w) + \alpha \min_v Q^*(f(x, u, w), v) \right\}.$$

I Q-Factors ottimali sono le uniche soluzioni di questo sistema di equazioni. Inoltre, i Q-factors ottimali possono essere ottenuti dall'algoritmo VI che scriveremo in maniera concisa come $Q_{k+1} = FQ_k$ dove F è l'operatore definito da

$$(FQ)(x, u) = E_w \left\{ g(x, u, w) + \alpha \min_v Q(f(x, u, w), v) \right\}, \quad \forall (x, u). \quad (2.28)$$

Abbiamo visto che VI con Q-factors converge a Q^* , per via della equivalenza con la versione con le cost-functions.

La versione originale dell'algoritmo di Q-Learning prevede un'approssimazione del valore atteso in 2.28. In particolare, viene generata una successione di coppie stato-controllo $\{(x_k, u_k)\}$ in base a qualche meccanismo probabilistico (non è richiesto che i termini della successione $\{x_k, u_k\}$ siano i.i.d.). Per ogni coppia, viene generato uno stato $x_{k+1} = f(x, u, w)$ in base alla funzione di transizione del sistema e la sua distribuzione di probabilità $P(\cdot|x, u)$. Il Q-Factor della coppia (x_k, u_k) viene aggiornato utilizzando uno stepsize $\gamma^i \in (0, 1]$, mentre tutti gli altri Q-Factor vengono lasciati uguali al passo precedente (si tratta dunque di un algoritmo asincrono):

$$Q_{k+1}(x, u) = (1 - \gamma^k)Q_k(x, u) + \gamma^k(F_k Q_k)(x, u) \quad \forall (x, u), \quad (2.29)$$

dove

$$(F_k Q_k)(x, u) = \begin{cases} g(x_k, u_k, w) + \alpha \min_v Q_k(f(x_k, u_k, w), v) & \text{se } (x, u) = (x_k, u_k) \\ Q_k(x, u) & \text{se } (x, u) \neq (x_k, u_k) \end{cases}. \quad (2.30)$$

Dunque $(F_k Q_k)(x_k, u_k)$ è una approssimazione del valore atteso in 2.28 effettuata tramite un singolo campione (u_k, x_k) .

Per garantire la convergenza dell'algoritmo 2.29 - 2.30 ai Q-Factors ottimali devono essere soddisfatte alcune condizioni. Innanzitutto ogni coppia (x_k, u_k) deve essere generata infinitamente spesso nella successione $\{(x_k, u_k)\}$. Oltre a questo, gli stati successivi ad x_k devono essere generati tramite la distribuzione $P(\cdot|x_k, u_k)$ in maniera indipendente. Ancora, come descritto nel paper *Technical Note, Q-learning* (Watkins, 1992), lo stepsize γ^k deve soddisfare

$$0 < \gamma_k < 1, \quad \forall k, \quad \sum_{k=0}^{\infty} \gamma_k = \infty, \quad \sum_{k=0}^{\infty} (\gamma_k)^2 < \infty.$$

Capitolo 3

Introduzione al CartPole Environment e formulazione del problema di Dynamic Programming associato

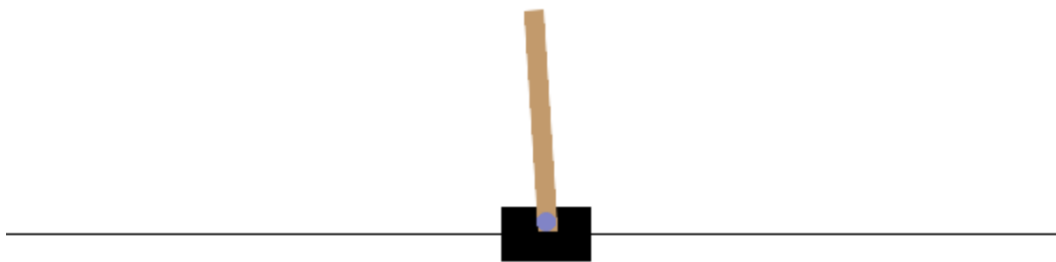


Figura 3.1. CartPole Environment (enviroment della libreria farama Gymnasium)

Il nostro primo obbiettivo è quello di risolvere tramite tecniche di DP il problema CartPole fornito dalla libreria Gymnasium (<https://gymnasium.farama.org/index.html>). L'ambiente è costituito da un carrello mobile su cui è attaccata un'asta, all'interno di uno spazio in cui il carrello è libero di muoversi a destra ed a sinistra. L'obbiettivo è riuscire a tenere in equilibrio l'asta fornendo i controlli al

$\mathbf{x}(m)$	$\dot{\mathbf{x}}(m/s)$	$\theta(rad)$	$\dot{\theta}(rad/s)$	control
$[-2.4, 2.4]$	$[-\infty, +\infty]$	$[-0.209, 0.209]$	$[-\infty, +\infty]$	{Left, Right}

Tabella 3.1. Lo spazio di stato legale e lo spazio dei controlli del CartPole Environment originale. Quando $\theta = 0 rad$ l'asta è perpendicolare al carrello e punta verso l'alto.

carrello. Nel seguito sono descritte le specifiche dell'environment originale (tabella 3.1) e di un environment personalizzato (tabella 3.2).

3.1 Descrizione del sistema

Il sistema originale è composto da uno spazio di stato di dimensione finita (4) a valori continui in degli intervalli dati. Lo spazio dei controlli è di dimensione finita (1) a valori discreti.

La durata standard di un episodio è 500 di istanti temporali, tale grandezza è modificabile. Il delta fra due istanti corrisponde ad un periodo di $0.2s$. Il sistema di rewarding assegna $+1$ ad ogni istante temporale, incluso l'istante finale.

$\mathbf{x}(m)$	$\dot{\mathbf{x}}(m/s)$	$\theta(rad)$	$\dot{\theta}(rad/s)$
$[-2.4, 2.4]$	$[-\infty, \infty]$	$[-0.418, 0.418]$	$[-\infty, \infty]$

Tabella 3.2. Limiti alle variabili di stato legali della versione customizzata dell'environment. Gli stati con valori all'infuori di questi intervalli sono stati terminanti.



Figura 3.2. CartPole con $\theta = \pi/2$ e $x = -1$

3.2 Definizione del problema di DP associato

Se tradotto in un problema di Dynamic Programming, il sistema introdotto può essere descritto mediante i seguenti punti fondamentali:

1. Problema ad orizzonte infinito,
2. deterministico,
3. stazionario,
4. con spazio di stato continuo,
5. con spazio dei controlli discreto,
6. con discount.

Sebbene il programma preveda un limite al numero di passi eseguibili, quest'ultimo è comunque modificabile. Inoltre, viene naturale descrivere il problema come problema ad orizzonte infinito. Il problema è stazionario, cioè $g_i = g_j$ ed $f_i = f_j$ per ogni $i, j = 1, 2, \dots$.

L'introduzione del fattore di discount $\gamma \in [0, 1)$ ci permette di utilizzare l'algoritmo di Q-Learning, come dalle argomentazioni del capitolo precedente.

3.2.1 Dinamica delle transizioni e dei guadagni

Per descrivere la dinamica degli stati terminanti¹, diciamo che se $\hat{x}$ è uno stato terminante allora

$$f(\hat{x}, u_k) = \hat{x} \quad e \quad g(\hat{x}, u_k) = 0 \quad \forall u_k, \quad k = 1, 2, \dots \quad (3.1)$$

Per ogni stato non terminante x ed ogni controllo u , si ha

$$g(x, u) = \begin{cases} 1 & \text{se } f(x, u) \text{ non è uno stato terminante} \\ -300 & \text{se } f(x, u) \text{ è uno stato terminante} \end{cases} \quad (3.2)$$

L'obiettivo è trovare una policy che massimizzi il guadagno.

3.3 Discretizzazione del sistema

Per semplificare il problema, introduciamo alcune approssimazioni che ci permetteranno di collocare il problema in una classe di problemi più facilmente risolvibili. Decidiamo di discretizzare lo spazio di stato.

Il nuovo spazio di stato discreto è composto da un numero finito di sottoinsiemi del precedente spazio di stato.

Adesso possiamo dare una descrizione del nuovo problema, mediante i seguenti punti fondamentali

1. Problema ad orizzonte infinito,
2. stocastico,

¹Ricordiamo che nell'implementazione del sistema all'interno di Gymnasium una volta raggiunto uno stato terminante il sistema viene resettato e si inizia una nuova iterazione del processo partendo dallo stage 1.

3. stazionario,
4. con spazio di stato discreto,
5. con spazio dei controlli discreto,
6. con discount.

Notiamo che la discretizzazione dello stato rende il problema stocastico. A questo punto, possiamo utilizzare l'algoritmo di Q-Learning per il calcolo di una policy che massimizzi il guadagno $J_\mu(x_0)$ per ogni stato iniziale x_0 . Tale policy si ottiene scegliendo $\mu(x) = \max_u Q^\pi(x, u)$ quando si sono ottenuti i Q-Factors Q^π tali che $Q^\pi \rightarrow Q^*$. Le massimizzazioni preservano la validità dell'algoritmo: la situazione equivalente si può ottenere cambiando il segno ai costi $g(x, u)$ e minimizzando nelle equazioni.

Nel capitolo precedente abbiamo definito l'algoritmo di Q-Learning per problemi ad orizzonte infinito stocastici, e abbiamo spiegato come questo sia un algoritmo model-free, cioè in cui non viene stimata la distribuzione di probabilità che regola le transizioni del sistema, ossia non viene determinata $f(x, u, w)$, quindi non c'è previsione dello stato futuro. I comandi vengono scelti in base ai valori attesi presenti nei Q-Factors.

Diamo nell'algoritmo 1 una prima versione dell'algoritmo in cui la fase di training viene effettuata su una successione di stati i.i.d., nell'algoritmo 2 un'implementazione in cui fra i termini della successione esistono rapporti causali (si segue la dinamica del sistema). Ricordiamo che l'unica condizione necessaria affinché i Q-Factors convergano ai Q-Factors ottimali Q^* è che ogni coppia stato-controllo ricorra nella successione $\{(x_k, u_k)\}$ anche quando $N \rightarrow \infty$, cioè non esiste una coppia (x_k, u_k) che non ricorra più da un certo N in poi. Utilizzeremo poi alcune tecniche tipiche del RL per ridurre i tempi di training. Quest'ultime fanno riferimento al fatto che siamo maggiormente interessati a determinare in maniera esatta i Q-values $Q(x, u)$ in cui u è un controllo ottimo piuttosto che i casi in cui u è non ottimo, per i quali possiamo accontentarci di stime più approssimative.

Algorithm 1: CartPole Q-Learning with i.i.d. states

Per l'implementazione si è in parte seguito *Balancing a CartPole System with Reinforcement Learning - A Tutorial* (Kumar, 2019).

```

1 initialize  $\mathbf{Q}(\text{state}, \text{action}) \leftarrow 0 \quad \forall (\text{state}, \text{action})$ .
2 discount factor  $\alpha = 0.99$ .
3 for  $i = 1, \dots, EPISODES$  do
4    $\gamma \leftarrow 1/i$ 
5   reset environment to random initial state and get observation.
6   state  $\leftarrow$  discretize (observation).
7   take random action, then observe and discretize next state.
8   if next state is a terminating state then
9     | reward = -3.
10  else
11    | reward = 1.
12  end
13   $\mathbf{Q}_i(\text{state}, \text{action}) \leftarrow (1 - \gamma)\mathbf{Q}_{i-1}(\text{state}, \text{action}) + \gamma(\text{reward} +$ 
     $\max_{\text{action}} \mathbf{Q}(\text{nextstate}, \text{action}))$ .
14 end
15 Store Q table.
```

Algorithm 2: CartPole Q-Learning

In questa versione dell'algoritmo utilizziamo una policy ϵ -greedy per la scelta del comando. Con probabilità ϵ si sceglie un comando random, con probabilità $1 - \epsilon$ si sceglie il comando massimizzante. Ad ogni iterazione del **for** si decrementa il valore di ϵ . Questo permette di velocizzare i tempi di convergenza ad una policy subottimale che approssimi in maniera sufficiente i Q-values massimizzanti.

La Q-table viene inizializzata a 0 in tutte le sue entrate, la prima scelta dell'azione per il Q-factor $Q(x, u)$ è casuale per ogni (x, u) .

Per l'implementazione si è in parte seguito *Balancing a CartPole System with Reinforcement Learning - A Tutorial* (Kumar, 2019).

```

1 initialize  $\mathbf{Q}(\text{state}, \text{action}) \leftarrow 0 \quad \forall (\text{state}, \text{action})$ .
2 discount factor  $\alpha = 0.99$ .
3 for  $i = 1, \dots, EPISODES$  do
4    $\gamma \leftarrow 1/i$ .
5   reset environment to random initial state and get observation.
6   state  $\leftarrow$  discretize (observation.)
7   while Episode is not terminated or truncated do
8     select action using  $\epsilon$ -greedy policy, then observe and discretize next
      state.
9     if next state is a terminating state then
10      | reward  $= -300$ .
11    else
12      | reward  $= 1$ .
13    end
14     $\mathbf{Q}_i(\text{state}, \text{action}) \leftarrow (1 - \gamma)\mathbf{Q}_{i-1}(\text{state}, \text{action}) + \gamma(\text{reward} +$ 
       $\max_{\text{action}} \mathbf{Q}(\text{nextstate}, \text{action}))$ .
15    state  $\leftarrow$  nextstate.
16  end
17 end
18 Store Q table.
```

Il programma dimostrativo utilizza la Q-table ottenuta tramite l'algoritmo 2. Al momento del reset lo stato iniziale viene scelto in maniera randomica all'interno degli intervalli rappresentati nella tabella 3.3, valori con i quali è garantito un alto rate di successi. Se si vuole, è possibile scegliere i valori di ciascuna variabile.

$\mathbf{x}(m)$	$\dot{\mathbf{x}}(m/s)$	$\theta(rad)$	$\dot{\theta}(rad/s)$
$[-1, 1]$	$[-0.5, 0.5]$	$[-0.3, 0.3]$	$[-0.5, 0.5]$

Tabella 3.3. Per stati iniziali a valori negli intervalli presentati nella tabella, si raggiunge lo stato di terminazione artificiale (500-esimo step) con probabilità $\approx 93\%$.

Problemi incontrati Ciò che vorremmo fare è estendere il problema, allargando l'intervallo della variabile θ ad $[-\infty, \infty]$. Per far ciò avremmo bisogno di rendere molto più "fine" la discretizzazione. La grande e invalidante limitazione che si è incontrata è stato però il fatto che il costo computazionale dell'algoritmo Q-learning

è esponenziale nella dimensione dello stato (Hastie T., Tibshirani R., Friedman J., 2009). Per questo motivo è stato necessario utilizzare altre tecniche più efficaci, discusse nei capitoli successivi.

3.4 Limitazioni nell'osservabilità e ricostruzione delle variabili di stato

Tipicamente, nei problemi di controllo non sempre è possibile monitorare in maniera diretta tutte le variabili di stato che si vorrebbero. Si è dunque deciso di limitare l'output del sistema, oscurando le variabili di stato $\dot{x}_i$ e $\dot{\theta}_i$. Per ampliare la dimensione dello spazio di stato, sono state ricostruite le variabili $\dot{x}_{i-1}$ e $\dot{\theta}_{i-1}$, che rappresentano rispettivamente la velocità all'istante precedente e la velocità angolare all'istante precedente, tramite i calcoli $\dot{x}_{i-1} = (x_i - x_{i-1})/T$ e $\dot{\theta}_{i-1} = (\theta_i - \theta_{i-1})/T$, dove $T = 0.02$ è il periodo che intercorre fra due istanti temporali.

Lo stato del sistema all'istante i è dunque una quadrupla del tipo:

$$s_i = (x_i, \dot{x}_{i-1}, \theta_i, \dot{\theta}_{i-1}). \quad (3.3)$$

Si vorrebbe a questo punto utilizzare l'algoritmo 2 con il nuovo spazio di stato. Si presenta però un problema: al reset non abbiamo la possibilità di calcolare le variabili $\dot{x}_{i-1}$ e $\dot{\theta}_{i-1}$, poichè non vi è alcuno stato precedente allo stato iniziale. Per ovviare al problema, la decisione del primo controllo da effettuare viene presa soltanto in base alle variabili x e θ . Per far ciò sono stati creati degli stati artificiali in cui le variabili non misurabili prendono dei valori costanti. L'implementazione è l'algoritmo 3.

Algorithm 3: CartPole Q-Learning con stato ricostruito

```

1 initialize  $\mathbf{Q}(\text{state}, \text{action}) \leftarrow 0 \quad \forall (\text{state}, \text{action})$ .
2 discount factor  $\alpha = 0.99$ .
3 for  $i = 1, \dots, EPISODES$  do
4    $\gamma \leftarrow 1/i$ .
5   reset environment to random initial state and get observation.
6   state  $\leftarrow$  discretize (observation.)
7   while Episode is not terminated or truncated do
8     if state is the initial state then
9       select action based only on  $x$  and  $\theta$ , using an  $\epsilon$ -greedy policy.
        Then observe and discretize next state.
10    else
11      select action based on complete all the state variables
         $(x_i, \dot{x}_{i-1}, \theta, \dot{\theta}_{i-1})$ , using an  $\epsilon$ -greedy policy. Then observe and
        discretize next state.
12    end
13    if next state is a terminating state then
14      reward = -300.
15    else
16      reward = 1.
17    end
18     $\mathbf{Q}_i(\text{state}, \text{action}) \leftarrow (1 - \gamma)\mathbf{Q}_{i-1}(\text{state}, \text{action}) + \gamma(\text{reward} +$ 
       $\max_{\text{action}} \mathbf{Q}(\text{nextstate}, \text{action}))$ .
19    state  $\leftarrow$  nextstate.
20     $\dot{x}_i = (x_i - x_{i-1})/T$ .
21     $\dot{\theta}_i = (\theta_i - \theta_{i-1})/T$ .
22  end
23 end
24 Store Q table

```

Capitolo 4

Architetture parametriche e Neural Networks¹

Introduciamo adesso i metodi e le architetture parametriche di approssimazione. Successivamente ci focalizzeremo sulle Neural Networks (NN).

4.1 Architetture parametriche di approssimazione

In questa sezione indagheremo su come dare una buona approssimazione delle cost functions $\tilde{J}_k$ tramite delle funzioni parametriche, dove si ricava l'approssimazione eseguendo l'ottimizzazione sui parametri (figura 4.1).

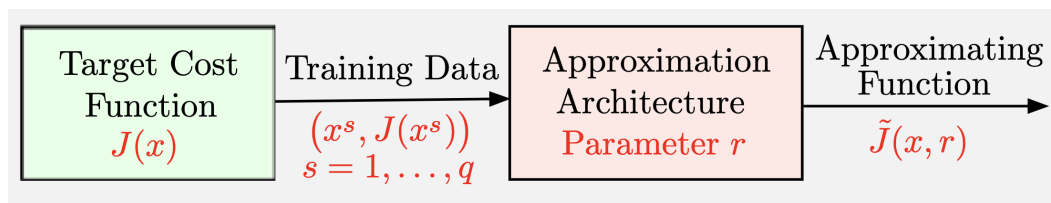


Figura 4.1. La struttura generale dell'approssimazione parametrica di una cost function. Si approssima $J(x)$ per mezzo di una funzione $\tilde{J}(x, r)$, dove r è un vettore di parametri. Utilizzando una grande mole di dati per il training $(x^s, J(x^s))$, $s = 1, \dots, q$ e cerchiamo un vettore di parametri $\hat{r}$ che minimizzi l'errore $J(x^s) - \tilde{J}(x^s, \hat{r})$, $s = 1, \dots, q$. (Fonte: Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific, p.331)

Le cost function approssimate sono scelte da una classe parametrica di funzioni $\tilde{J}(x_k, r_k)$, chiamata *approximation architecture* dove r_k è un vettore di parametri scalari.

4.1.1 Architetture Feature-Based

Una tipologia rilevante di cost approximation coinvolge un processo chiamato *feature extraction* (Bertsekas, 2023), dove ogni stato x_k viene mappato in un

¹Il presente capitolo segue fedelmente nella trattazione degli argomenti, inclusi teoremi, dimostrazioni, esempi il testo Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific. e Bertsekas D.P. (2019), *Reinforcement Learning and Optimal Control*, Belmont, Massachusetts, Athena Scientific.

vettore $\phi_k(x_k)$ chiamato feature vector associato ad x_k al tempo k , che possiamo rappresentare come

$$\phi_k(x_k) = (\phi_{1,k}(x_k), \dots, \phi_{m_k,k}(x_k))$$

e di cui chiamiamo *features* le componenti. Una approssimazione feature-based ha dunque la forma

$$\tilde{J}_k(x_k, r_k) = \hat{J}_k(\phi_k(x_k), r_k),$$

dove r_k è un vettore di parametri e $\hat{J}_k$ una qualche funzione.

Architetture feature-based lineari

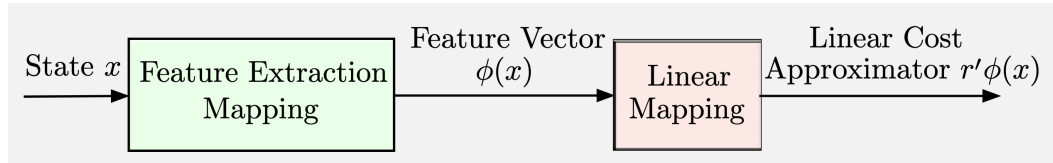


Figura 4.2. Esempio di un'architettura feature-base lineare. Utilizziamo la mappa $\phi_k(x_k)$ come input di una trasformazione lineare definita da $r'\phi_k(x_k)$. (Fonte: Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific, p.334)

Nella figura 4.2 vediamo la pipeline di una architettura feature-based lineare. Le componenti del vettore r vengono spesso chiamate *weights*. Per dare un'intuizione riguardo l'applicazione delle feature functions, immaginiamo una cost function J_k^* non lineare e complicata. Possiamo suddividerla in diversi "pezzi" meno complicati e in particolare, se le features codificano bene le non linearità di J_k^* , possiamo utilizzare un'architettura lineare $\hat{J}_k$ semplice per apporssimare J_k^* . In questo caso, la funzione $\hat{J}(\phi_k(x_k), r_k) = r'\phi_k(x_k)$ può anche essere vista come un elemento di $\text{Span}\{\phi_{1,k}(x_k), \dots, \phi_{m_k,k}(x_k)\}$.

Un esempio semplice ma illustrativo esempio (tratto da *A Course in Reinforcement Learning* (Bertsekas, 2023)) si ottiene scegliendo

$$\phi_l(x) = \begin{cases} 1 & \text{se } x \in S_l \\ 0 & \text{se } x \notin S_l \end{cases} \quad l = 1, \dots, m$$

dove $S_1, \dots, S_m$ definiscono una partizione dello spazio di stato.

Per problemi stazionari con orizzonti infiniti è comune utilizzare le stesse features e parametri per ogni stage. La funzione $\tilde{J}(x, r) = r'\phi(x)$ è una funzione a tratti costante che, una volta trovato il vettore r , dà una approssimazione della target function (cfr. figura 4.3).

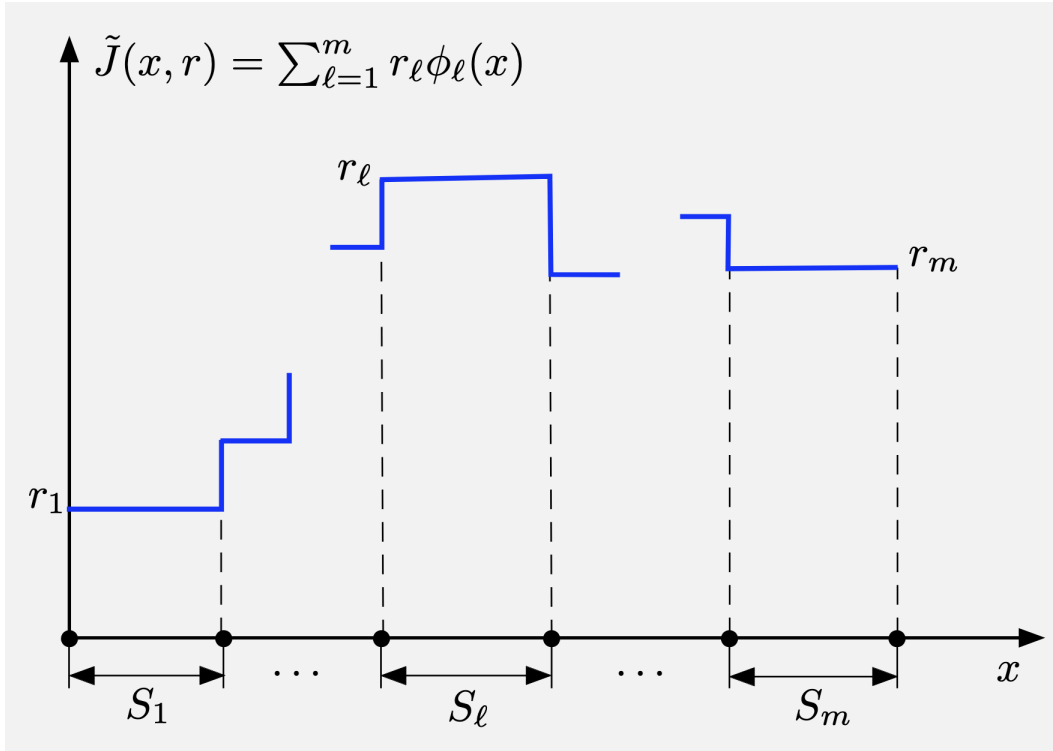


Figura 4.3. Approssimazione lineare. (Fonte: Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific, p.335)

4.1.2 Training di architetture lineari e non lineari

Discutiamo brevemente il processo di training per la scelta del parametro r di un'architettura parametrica $\tilde{J}(x, r)$. In particolare, trattiamo i metodi incrementali che utilizzano gradiente. Un tipo molto comune di approssimazione è basato su una minimizzazione ai quadrati minimi, che adesso introduciamo. Dato un insieme di coppia stato-costo (x^s, β^s) , $s = 1, \dots, q$, chiamato training set, determiniamo il vettore di parametri r risolvendo il seguente problema

$$\min_r \sum_{s=1}^q (\tilde{J}(x^s, r) - \beta^s)^2, \quad (4.1)$$

dove $\tilde{J}(x, r)$ è un'approssimazione della target cost function J e $\beta^s = J(x^s) + w$ dove w è un disturbo o errore. Se l'architettura è lineare, cioè $\tilde{J}(x, r) = r' \phi(x)$, allora la 4.1 è convessa ed esiste una soluzione analitica a formula chiusa. Il problema assume infatti la forma

$$\min_r \sum_{s=1}^q (r' \phi(x^s) - \beta^s)^2.$$

Calcoliamo dove si annulla il gradiente:

$$\nabla \min_r \sum_{s=1}^q (r' \phi(x^s) - \beta^s)^2 = \min_r \sum_{s=1}^q \nabla (r' \phi(x^s) - \beta^s)^2,$$

calcolando il gradiente dell' s -esimo termine quadratico

$$\nabla(r'\phi(x^s) - \beta^s)^2 = 2 \begin{pmatrix} (r'\phi(x^s) - \beta^s)\phi_1(x^s) \\ (r'\phi(x^s) - \beta^s)\phi_2(x^s) \\ \vdots \\ (r'\phi(x^s) - \beta^s)\phi_n(x^s) \end{pmatrix} = 2(r'\phi(x^s) - \beta^s)\phi(x^s),$$

da cui otteniamo che il gradiente è nullo se e solo se

$$\sum_{s=1}^q \phi(x^s)(r'\phi(x^s) - \beta^s) = 0,$$

oppure equivalentemente

$$\sum_{s=1}^q \phi(x^s)\phi(x^s)'r = \sum_{s=1}^q \phi(x^s)\beta^s.$$

Ammettendo che la matrice $\sum_{s=1}^q \phi(x^s)\phi(x^s)'$ sia invertibile, otteniamo che il vettore di parametri minimizzante è

$$\hat{r} = \left(\sum_{s=1}^q \phi(x^s)\phi(x^s)' \right)^{-1} \sum_{s=1}^q \phi(x^s)\beta^s.$$

Se la matrice non è invertibile, si somma una funzione quadratica in r , detta *regularization function*, alla funzione ai quadrati minimi.

Le argomentazioni per questa sottosezione sono tratte da *A Course in Reinforcement Learning* (Bertsekas, 2023).

Dunque, un'architettura lineare apporta l'importante vantaggio di poter risolvere la minimizzazione con una formula chiusa. Le architetture non lineari (come ad esempio le NN) possono però essere molto più precise nell'approssimare la funzione target. Se si utilizzano architetture di questo tipo, il problema di minimizzazione associato è non convesso ed è dunque più difficile da risolvere. In questi casi entrano in gioco gli algoritmi di discesa del gradiente.

Discesa del gradiente e metodo incrementale

Discuteremo adesso dei metodi popolari per la soluzione di problemi ai quadrati minimi (4.1) utilizzando architetture non lineari, dove assumiamo che l'architettura sia differenziabile rispetto al vettore parametrico.

Pensiamo al problema 4.1 come alla minimizzazione di una somma di funzioni componente

$$f(y) = \sum_{i=1}^m f_i(y), \quad (4.2)$$

dove ogni f_i è un operatore lineare ad n argomenti, dove n è la dimensione del vettore y (vettore parametro).

Theorem 9 (Algoritmo di discesa del gradiente (Bertsekas, 2019)). *L'algoritmo di discesa del gradiente ordinario per la minimizzazione di problemi del tipo 4.2 genera*

una successione $\{y_k\}$, partendo da un qualche y_0 predeterminato. L'iterazione ha la seguente forma

$$y_{k+1} = y_k - \gamma_k \nabla f(y_k) = y_k - \gamma_k \sum_{i=1}^m \nabla f_i(y_k), \quad (4.3)$$

dove $\gamma_k > 0$ è lo stepsize, ovvero la quantità dello spostamento nella direzione del gradiente, verso opposto.

Theorem 10 (Algoritmo incrementale di discesa del gradiente (Bertsekas, 2019)). *Il metodo incrementale di discesa del gradiente è simile a quello ordinario, ma in ciascuna iterazione si utilizza il gradiente di una singola componente di f , ossia una delle f_i nella 4.2. L'iterazione ha la seguente forma*

$$y_{k+1} = y_k - \gamma_k \nabla f_{i_k}(y_k), \quad (4.4)$$

dove $i_k \in \{1, \dots, m\}$ è l'indice della componente scelta per l'iterazione k .

Intuitivamente, nella versione incrementale stiamo minimizzando il problema ai quadrati minimi con la target function calcolata in un singolo data point. Sebbene sia una semplificazione radicale, l'algoritmo incrementale funziona sorprendentemente bene ed è computazionalmente molto meno dispendioso del metodo ordinario. Nella pratica, l'utilizzo di questo algoritmo può essere molto articolato a seconda del problema che si sta affrontando.

4.2 Neural Networks

Nella pratica, è spesso impossibile conoscere un set di features adeguate a risolvere il problema che si vuole affrontare. Esistono però metodi automatici di costruzione delle features. Il metodo più popolare fra questi sono le *neural networks*.

Le NN hanno uno spettro di impiego molto ampio. In questa sezione ci concentriamo sul loro utilizzo per problemi di DP ad orizzonte finito. Nei capitoli successivi estenderemo il loro utilizzo a problemi ad orizzonte infinito e in particolare definiremo delle tecniche per risolvere il problema CartPole.

Come esempio di utilizzo nel contesto della DP ad orizzonte finito, possiamo impiegare una NN per approssimare una cost function J_k^* . In particolare, possiamo impiegare la NN per calcolare J_{N-1}^* , poi J_{N-2}^* e procedere a ritroso, esattamente come nell'algoritmo di DP.

Adesso introduciamo il più diffuso tipo di NN per RL, ossia il *multilayer perceptron*.

Per descrivere le NN nel contesto della DP ad orizzonte finito, poniamoci al generico stage k , dove vogliamo approssimare una target function J_k . Per comodità, omettiamo i pedici k e consideriamo l'architettura parametrica

$$\tilde{J}(x, v, r) = r' \phi(x, v) \quad (4.5)$$

che dipende da due vettori di parametri, v ed r . L'obiettivo è scegliere questi due vettori in modo che $\tilde{J}(x, v, r)$ approssimi la target function. Questo viene fatto collezionando un grande numero di campioni stato-cost (x^s, β^s) , $s = 1, \dots, q$, e trovando la giusta funzione $\tilde{J}(x, v, r)$ della forma 4.5. Cioè dobbiamo trovare la coppia (v, r) che minimizzi

$$\sum_{s=1}^q (\tilde{J}(x^s, v, r) - \beta^s)^2.$$

Qui si ha che il parametro v parametrizza $\phi(x, v)$, che possiamo in un certo senso vedere come un feature vector, mentre r è un vettore parametrico di pesi lineari per le componenti di $\phi(x, v)$.

4.2.1 Single Layer Perceptron

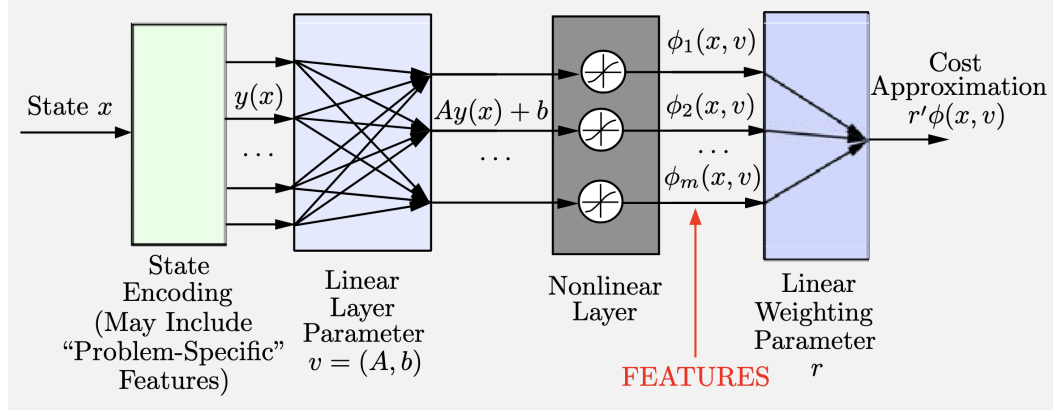


Figura 4.4. Illustrazione del single layer perceptron. Una volta codificato lo stato x come vettore $y(x)$, si applicano sequenzialmente una trasformazione affine ed una funzione non-lineare element-wise, $\phi_l(x, v) = \sigma((Ay(x) + b)_l)$. Come ultimo step, si effettua il prodotto scalare con il vettore di parametri r . (Fonte: Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific, p.353)

Una neural network è una architettura parametrica che fornisce una classe di funzioni parametriche $\tilde{J}(x, v, r)$ della forma 4.5. La tipologia basilare di NN è chiamata single layer perceptron. Lo stato x viene codificato come un vettore $y(x) = (y_1(x), \dots, y_n(x))$, a cui viene poi applicata una trasformazione affine

$$Ay(x) + b,$$

Questa trasformazione è chiamata *linear layer* della NN. Consideriamo le entrate di A e b come parametri da determinare, e li raggruppiamo nel vettore parametrico $v = (A, b)$.

Ciascuno degli m output scalari del linear layer (le entrate del vettore risultato della trasformazione lineare)

$$(Ay(x) + b)_l, \quad l = 1, \dots, m,$$

viene dato in input ad una funzione non-lineare differenziabile monotona crescente $\sigma : \mathbb{R} \rightarrow \mathbb{R}$. Un candidato popolare per tale funzione non-lineare è la *rectified linear unit* (abbreviata ReLU, cfr. figura 4.5), che è la funzione $\max\{0, \xi\}$, approssimata da una funzione σ differenziabile come ad esempio $\sigma(\xi) = \ln(1 + e^\xi)$.

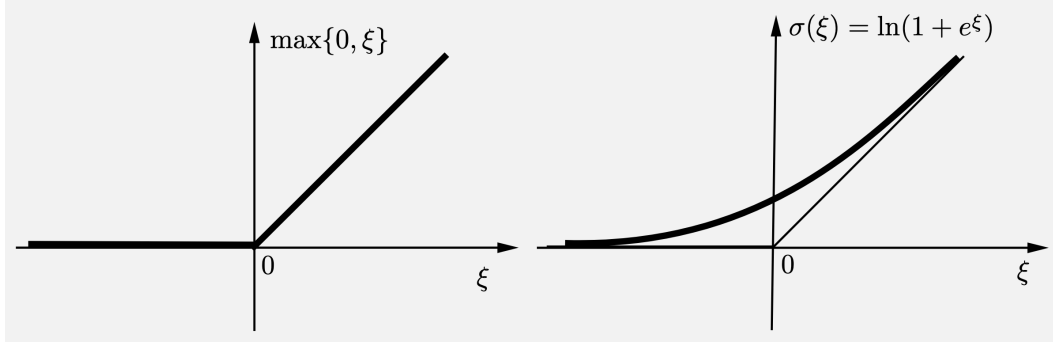


Figura 4.5. A destra la ReLU, che possiamo pensare come una versione con una curva più morbida (differenziabile) della funzione $\max\{0, \xi\}$. (Fonte: Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific, p.354)

Ci si riferisce alla funzione σ come "nonlinear unit" e al corrispondente layer come "nonlinear layer". L'output delle non linear units sono gli scalari

$$\phi_l(x, v) = \sigma((Ay(x) + b)_l), \quad l = 1, \dots, m.$$

Una possibile interpretazione è vedere $\phi_l(x, v)$ come le features di x , che vengono linearmente combinate utilizzando i pesi $r_l, l = 1, \dots, m$, per produrre il risultato finale

$$\tilde{J}(x, v, r) = \sum_{l=1}^m r_l \phi_l(x, v) = \sum_{l=1}^m r_l \sigma((Ay(x) + b)_l). \quad (4.6)$$

4.2.2 Training delle Neural Networks

Dato un training set di coppie stato-costo $(x^s, \beta^s), s = 1, \dots, q$, i parametri della NN A, b, r si ottengono risolvendo il problema di minimizzazione

$$\min_{A, b, r} \sum_{s=1}^q \left(\sum_{l=1}^m r_l \sigma((Ay(x^s) + b)_l) - \beta^s \right)^2. \quad (4.7)$$

In maniera più concisa possiamo scrivere

$$\min_{A, b, r} \sum_{s=1}^q \left(r' \sigma((Ay(x^s) + b)) - \beta^s \right)^2,$$

ma la 4.7 evidenzia meglio come gli r_l siano pesi relativi a features $\phi_l(x, v) = \sigma((Ay(x^s) + b)_l)$ distinte.

Nella pratica, è comune aumentare la cost function 4.7 con una *regularization* function, come ad esempio dei termini quadratici nei parametri A, b, r . Nel contesto delle NN, la regolarizzazione è fondamentale per evitare l'*overfitting*, un fenomeno che avviene quando il numero di parametri della NN è relativamente grande (di dimensione prossima a quella del training set). In casi come questo, il modello potrebbe ottenere ottimi risultati sul training set, ma non con dati nuovi.

Un altro grande problema è scegliere come risolvere la 4.7. Il numero di componenti q potrebbe essere molto alto; in tali casi i metodi incrementali visti precedentemente diminuiscono notevolmente i tempi di computazione. Approfondiremo altri metodi a breve.

Una NN è dunque in grado di produrre le feature corrette per approssimare qualunque funzione tramite la combinazione di queste features, cfr. 4.6. Per farci un'idea di come questo sia possibile, seguiamo la seguente illustrazione. Notiamo che è sufficiente un singolo layer non lineare, preceduto e seguito da un layer lineare rispettivamente.

Consideriamo le features si una variabile scalare x prodotte mediante una Neural Network che utilizza come funzione non lineare la ReLU

$$\sigma(\xi) = \max\{0, \xi\}$$

preceduta da una funzione lineare

$$L(x) = \gamma(x - \beta)$$

La composizione di queste produce la feature

$$\phi_{\beta,\gamma}(x) = \max\{0, \gamma(x - \beta)\}, \quad (4.8)$$

che possiamo visualizzare nella figura 4.6. Combinando due feature della forma 4.8

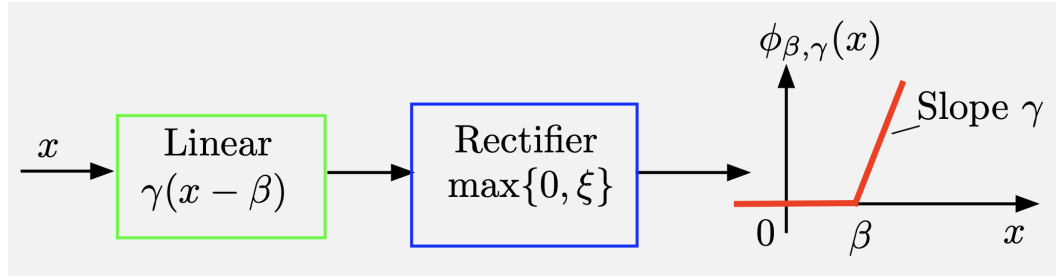


Figura 4.6. La feature $\phi_{\beta,\gamma}(x)$. (Fonte: Bertsekas D.P. (2019), *Reinforcement Learning and Optimal Control*, Belmont, Massachusetts, Athena Scientific, p.156)

con la stessa pendenza γ e traslazioni diverse, otteniamo la nuova feature

$$\phi_{\beta_1,\beta_2,\gamma}(x) = \phi_{\beta_1,\gamma}(x) - \phi_{\beta_2,\gamma}(x).$$

Se ancora combiniamo due features della forma appena ottenuta, otteniamo la feature pulsazione che possiamo utilizzare come blocco fondamentale

$$\phi_{\beta_1,\beta_2,\beta_3,\beta_4,\gamma}(x) = \phi_{\beta_1,\beta_2,\gamma}(x) - \phi_{\beta_3,\beta_4,\gamma}(x), \quad (4.9)$$

possiamo cioè ottenere qualunque feature come combinazione lineare di pulsazioni. Ovviamente, per ottenere una pulsazione è sufficiente un solo layer non lineare, per cui anche con l'architettura più semplice di NN si ha una grandissima potenza di approssimazione, cfr. figura 4.7.

4.2.3 Backpropagation

Consideriamo di nuovo il training problem per le Neural Networks, che ha la forma:

$$\min_{v,r} \sum_{s=1}^q \left(\sum_{l=1}^m r_l \phi_l(x^s, v) - \beta^s \right)^2,$$

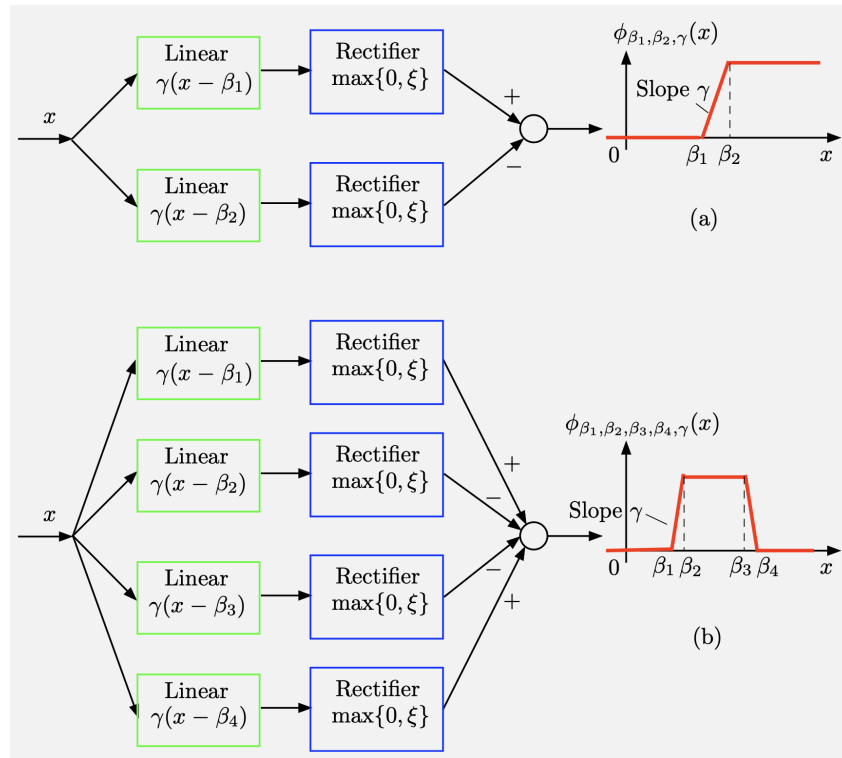


Figura 4.7. la combinazione di due features della forma $\phi_{\beta, \gamma}(x)$ sopra, sotto la feature pulsazione. (Fonte: Bertsekas D.P. (2023), *A Course in Reinforcement Learning*, Belmont, Massachusetts, Athena Scientific, p.157)

dove v rappresenta la collezione di tutti i parametri della rete e $\phi_l(x, v)$ è la l -esima feature prodotta come output della rete. Un metodo tipico per risolvere questo tipo di problema è l'utilizzo del gradiente, e in particolare dei metodi incrementali discussi nei capitoli precedenti. Adesso forniamo il risultato fondamentale per cui il gradiente rispetto ai pesi v di ogni feature $\phi_l(x, v)$ può essere calcolato in maniera efficiente tramite una tecnica chiamata *backpropagation* (Bertsekas, 2023).

Per dare una rappresentazione compatta della NN, denotiamo con $L_1, \dots, L_{m+1}$ le matrici che rappresentano i layer lineari. Denotiamo con $L_k \xi$, ($k > 1$) l'output del k -esimo layer dove ξ è l'output del precedente layer nonlineare. Denotiamo $\Sigma_1, \dots, \Sigma_m$ le mappe che rappresentano i layer nonlineari. Se l'input del k -esimo layer ($k > 1$) è il vettore y con componenti $y(j)$, denotiamo con $\Sigma_k y$ il vettore di output del k -esimo layer che ha componenti $\sigma(y(j))$. L'output finale della rete si può rappresentare come

$$F(L_1, \dots, L_{m+1}, x) = L_{m+1} \Sigma_m L_m \cdots \Sigma_1 L_1 x.$$

Adesso, possiamo scrivere l'errore quadratico fra l'output della rete e l'output desiderato come

$$E(L_1, \dots, L_{m+1}) = \frac{1}{2} (y - F(L_1, \dots, L_{m+1}, x))^2.$$

La derivata parziale di $E(L_1, \dots, L_k)$ rispetto ad $L_k(i, j)$, ossia la componente ij della matrice L_k , è data da

$$\frac{\partial E(L_1, \dots, L_{m+1})}{\partial L_k(i, j)} = -e' L_{m+1} \bar{\Sigma}_m L_m \cdots L_{k+1} \bar{\Sigma}_k I_{ij} \Sigma_{k-1} L_{k-1} \cdots \Sigma_1 L_1 x, \quad (4.10)$$

dove

$$e = y - F(L_1, \dots, L_{m+1}, x),$$

ed $\bar{\Sigma}_n, n = 1, \dots, m$, rappresenta la matrice diagonale con i termini sulla diagonale uguali alle derivate delle funzione n -esima funzione nonlineare σ calcolata negli opportuni punti, mentre I_{ij} è la matrice ottenuta da L_k ponendo a 0 ogni sua componente eccetto la componente ij che viene posta uguale ad 1. Attraverso il seguente algoritmo, si possono ottenere tutte le derivate parziali in 4.10:

1. Si effettua un calcolo attraversando la rete "in avanti" per calcolare sequenzialmente gli output dei layer lineari

$$L_1 x, L_2 \Sigma_1 L_1 x, \dots, L_{m+1} \Sigma_m L_m \cdots \Sigma_1 L_1 x.$$

Questo serve a sapere in quali punti bisogna calcolare le derivate nelle matrici $\bar{\Sigma}_n$ e per ottenere il vettore $e = y - F(L_1, \dots, L_{m+1}, x)$.

2. Si effettua un calcolo attraversando la rete "all'indietro" per calcolare sequenzialmente i termini

$$e' L_{m+1} \bar{\Sigma}_m L_m \cdots L_{k+1} \bar{\Sigma}_k$$

della formula 4.10, partendo da $e' L_{m+1} \bar{\Sigma}_m$ e continuando fino a $e' L_{m+1} \bar{\Sigma}_m \cdots L_2 \Sigma_1$.

Le argomentazioni presentate sono tratte da Bertsekas D.P. (2019), *Reinforcement Learning and Optimal Control*, Belmont, Massachusetts, Athena Scientific, p. 159.

4.3 Deep Q Network²

Quello che vogliamo fare adesso è proporre una soluzione al problema CartPole tramite i Q-factors, ma utilizzando le NN. Utilizzando una NN vogliamo dare una approssimazione $Q(s, a; \theta) \approx Q^*(s, a)$ della Q-function ottimale, anziché stimare la funzione Q^* separatamente su ogni input, come facevamo utilizzando l'algoritmo di Q-Learning.

Dunque definiamo con $Q(s, a; \theta)$ la nostra architettura di approssimazione parametrica. Nel caso generale, definiamo la loss function, ovvero la funzione che dobbiamo minimizzare rispetto a θ per ottenere l'approssimazione della target function, ad ogni iterazione i come

$$L_i(\theta_i) = E_{(x,u) \sim P(\cdot)} \left\{ (Q_i^*(x, u) - Q(x, u; \theta_i))^2 \right\}, \quad (4.11)$$

dove

$$Q_i^*(x, u) = E_w \left\{ g(x, u, w) + \alpha \max_v Q(f(x, u, w), v; \theta_{i-1}) \right\}. \quad (4.12)$$

Cioè la target function è data dal valore atteso reward ottenuto dall'applicazione del controllo u sommato al Q-value massimo dello stato futuro, dove come Q-function utilizziamo l'approssimazione dell'iterazione precedente $Q(x, u; \theta_{i-1})$. La probabilità $P(x, u)$ è invece una distribuzione di probabilità sull'insieme di coppie stato-controllo.

Il valore atteso nella 4.12 è nel nostro caso una generalizzazione, in quanto il sistema CartPole è deterministico.

I parametri dell'iterazione precedente θ_{i-1} vengono mantenuti fissati mentre si ottimizza la 4.11. Notiamo che le funzioni target 4.12 dipendono dai parametri dell'architettura.

Restringendoci al caso specifico del problema CartPole, riscriviamo la loss function liberandoci dei valori attesi:

$$L_i(\theta_i) = \left[Q_i^*(x_i, u_i) - Q(x_i, u_i; \theta_i) \right]^2, \quad (4.13)$$

dove (x_i, u_i) è la coppia stato-controllo dell'attuale iterazione e

$$Q_i^*(x_i, u_i) = \begin{cases} g(x_i, u_i) + \alpha \max_v Q(f(x_i, u_i), v; \theta_{i-1}) & \text{se } x_i \text{ è non-terminale} \\ g(x_i, u_i) & \text{se } x_i \text{ è terminale} \end{cases}. \quad (4.14)$$

A questo punto non ci resta che minimizzare l'equazione 4.13 rispetto a θ_i e aggiornare il valore θ_{i+1} in qualche modo. Esploriamo due possibili soluzioni.

Ottimizzazione per mezzo del gradiente Una possibilità è, assumendo che 4.13 sia differenziabile, calcolarne il gradiente a rispetto a θ_i e aggiornare θ_{i+1} muovendosi nella direzione del gradiente di una certa quantità *stepsize*. In effetti, 4.13 è differenziabile: è argomentato nei capitoli precedenti che 4.2 è combinazione di funzioni differenziabili, nel nostro caso $f = Q(x, u, \theta_i)$. Il termine $Q(x, u, \theta_{i-1})$ è a sua volta differenziabile in quanto ottenuto in maniera analoga a $Q(x, u, \theta_i)$.

²L'algoritmo DQN è stato per la prima volta presentato in Mnih V. et al. (2013), *Playing Atari with Deep Reinforcement Learning*, arXiv:1312.5602 e Mnih V. et al. (2015), *Human-level control through deep reinforcement learning*, Nature, 518:529-533. Le argomentazioni riportate in questa sezione seguono questi due articoli scientifici.

Il gradiente della 4.13 è

$$\nabla_{\theta_i} L_i(\theta_i) = \left[\left(g(x_i, u_i) + \gamma \max_v Q(x_i, u_i; \theta_{i-1}) - Q(x, u; \theta_i) \right) \nabla_{\theta_i} Q(x, u; \theta_i) \right] \quad (4.15)$$

Aggiorniamo quindi i parametri dell'architettura muovendoci di uno stepsize γ nella direzione opposta a 4.15:

$$\theta_{i+1} = \theta_i - \gamma \nabla_{\theta_i} L_i(\theta_i). \quad (4.16)$$

Per l'implementazione in python si è utilizzata la libreria Pytorch[?].

Ottimizzazione per mezzo di metodi senza gradiente Un'altra possibilità è minimizzare la 4.13 per mezzo di metodi che non coinvolgono il calcolo del gradiente. In questo caso, una volta ottenuto il parametro $\eta_i = \min_{\theta} L_i(\theta_i)$, aggiorniamo i parametri mediante

$$\theta_{i+1} = (1 - \gamma)\theta_i + \gamma\eta_i \quad (4.17)$$

con $\gamma \in (0, 1)$.

Capitolo 5

Utilizzo di DQN per CartPole e altri problemi di controllo

Diamo adesso una prima soluzione del problema CartPole tramite l'utilizzo di una Neural Network. Come prima implementazione, utilizziamo una multilayer perceptron con un singolo layer nonlineare. Per l'ottimizzazione della loss function utilizziamo il metodo del gradiente di cui abbiamo fornito una ampia descrizione (algoritmo 4) oppure, alternativamente, metodi gradient-free forniti dalla libreria Nevergrad (<https://facebookresearch.github.io/nevergrad/>) (algoritmo 5). In un momento successivo, risolviamo il problema con spazio di stato ad osservabilità ridotta.

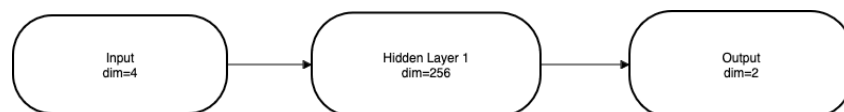


Figura 5.1. Architettura della NN utilizzata. Un singolo layer nonlineare (ReLU) composto da 256 neuroni.

Per migliorare i tempi di training possiamo anche utilizzare NN con più layer nonlineari, una pratica comune.

5.1 Problemi riscontrati e soluzioni adottate

Negli algoritmi Temporal Difference (TD) come il Q-learning e di conseguenza il DQN, l'update del costo (nel nostro caso il Q-value) per un certo stato (coppia stato-controllo), genera il rischio di cambiare inopportuno il costo di altri stati, compreso lo stato su quale si sta facendo bootstrapping, ossia lo stato successivo in base al cui costo stiamo aggiornando il costo dello stato attuale. Negli algoritmi off-policy come il DQN, questo può generare seri problemi, come la divergenza dei parametri di learning¹.

Nei test fatti sul sistema CartPole, in qualunque versione del problema, ma in particolare nella versione ad osservabilità limitata, si riscontrano oscillazioni nella durata degli episodi anche dopo lunghi periodi di stabilità. Non possiamo contare su alcuna proprietà di convergenza, proprietà che avevamo invece viste verificate nel caso dell'algoritmo di Q-learning, discusso nei capitoli precedenti.

¹come descritto in an Hasselt H., Hessel M., Doron Y., Sonnerat N., Strub F., Modayil J. (2018), *Deep Reinforcement Learning and the Deadly Triad*, arXiv:1812.02648.

Algorithm 4: CartPole with Neural Network using gradient descent.

Per l'implementazione si è seguito [5], [4] e [8].

```

1 create neural network for approximating  $Q$ .
2 initialize weights to random values.
3 discount factor  $\alpha = 0.99$ .
4 for  $i = 1, \dots, EPISODES$  do
5   reset environment to random initial state and get observation.
6   while Episode is not terminated or truncated do
7     select action using  $\epsilon$ -greedy policy, then observe next state.
8     if next state is a terminating state then
9       | reward =  $-300$ .
10    else
11      | reward =  $1$ .
12    end
13
14     $y_j = \begin{cases} \text{reward} & \text{if next state is terminal} \\ \text{reward} + \alpha \max_u Q(\text{nextstate}, u; \theta) & \text{if not terminal} \end{cases}$ 
15    Perform gradient descent step on  $(y_j - Q(\text{state}, \text{action}; \theta))^2$  using
16    end
17 end

```

Nella figura 5.2 vediamo le importanti oscillazioni nella durata dell'episodio. Analizziamo adesso il motivo di questo comportamento, poi discutiamo alcune possibili miglorie per offrire maggiore stabilità all'algoritmo.

5.1.1 Deadly Triad²

In *Reinforcement Learning, An Introduction, Second Edition* (Sutton & Barto, 2018) la combinazione all'interno di uno stesso algoritmo di off-policy learning, bootstrapping e approssimazione della cost function viene chiamata la "triade mortale" (deadly triad) perchè porta a possibile divergenza (una cost function potrebbe assumere un valore sempre maggiore fino a divergere). Nel caso del DQN, sebbene rientri negli algoritmi di questo tipo, la divergenza non avviene: i Q-values non assumono mai valori spropositati fino a divergere, ed è anzi un algoritmo che funziona molto bene in diverse situazioni³. Comunque, come mostrato nelle figure, si registrano diverse oscillazioni nelle performance dell'algoritmo. Per cogliere il motivo di tale comportamento, riportiamo un esempio ben noto e di facile comprensione, anche questo tratto da *Deep Reinforcement Learning and the Deadly Triad*⁴.

²Le argomentazioni di questa sezione seguono van Hasselt H., Hessel M., Doron Y., Sonnerat N., Strub F., Modayil J. (2018), *Deep Reinforcement Learning and the Deadly Triad*, arXiv:1812.02648.

³van Hasselt H., Hessel M., Doron Y., Sonnerat N., Strub F., Modayil J. (2018), *Deep Reinforcement Learning and the Deadly Triad*. arXiv:1812.02648.

⁴vedi nota 3

Algorithm 5: CartPole with Neural Network using gradient-free methods .

Per l'implementazione si è seguito [5], [4] e [8].

```

1 create neural network for approximating  $Q$ .
2 initialize weights to random values.
3 discount factor  $\alpha = 0.99$ .
4 for  $i = 1, \dots, EPISODES$  do
5   reset environment to random initial state and get observation.
6   while Episode is not terminated or truncated do
7     select action using  $\epsilon$ -greedy policy, then observe next state.
8     if next state is a terminating state then
9       | reward = -300.
10    else
11      | reward = 1.
12    end
13
14      
$$y_j = \begin{cases} \text{reward} & \text{if next state is terminal} \\ \text{reward} + \alpha \max_u Q(\text{nextstate}, u; \theta) & \text{if not terminal} \end{cases}$$

15      use Nevergrad to minimize eq. 4.13.
16      Update weights by
17      
$$\theta_{i+1} = (1 - \gamma)\theta_i + \gamma\eta_i$$

18      where  $\eta_i$  are the minimizing weights.
19      state  $\leftarrow$  nextstate.
20    end
21 end

```

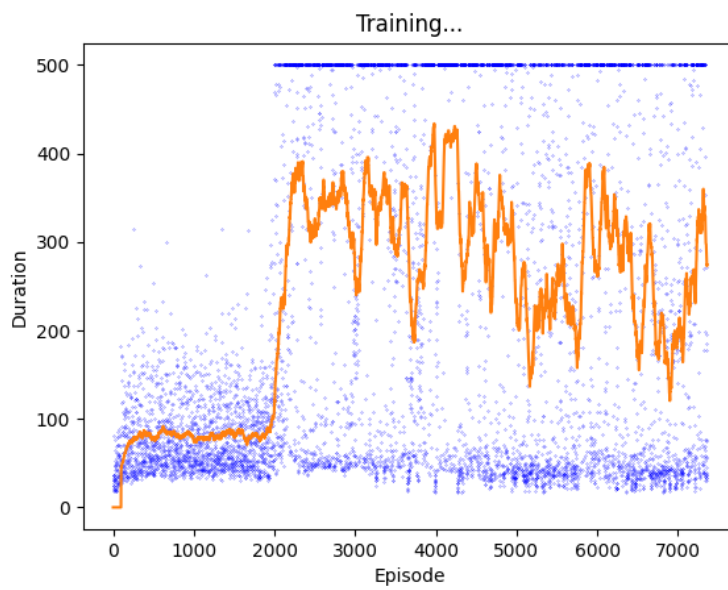


Figura 5.2. Il caso trattato utilizza DQN standard. In blu la durata degli episodi per il problema CartPole con stato iniziale randomico nel range ($x \in [-0.05, 0.05]$, $\dot{x} \in [-0.05, 0.05]$, $\theta \in [-0.628, 0.628]$, $\dot{\theta} \in [-0.05, 0.05]$), senza variabili oscure. È evidenziata una importante oscillazione nella durata degli episodi. In arancione la durata media degli ultimi 100 episodi. La rete utilizzata è quella illustrata nella figura 5.1. Notiamo come da un certo punto in poi la maggior parte degli episodi hanno durata 500 (il massimo), ma non si ha convergenza.

Intuizione della dinamica di learning degli algoritmi TD Riportiamo quanto spiegato in *Deep Reinforcement Learning and the Deadly Triad* (van Hasselt et al., 2018), per il caso più generale degli algoritmi di Temporal Difference Learning, all'interno dei quali rientrano anche il Q-learning e il DQN. L'obiettivo del TD learning è di aggiornare i pesi w di una value function v_w per renderla prossima ad una funzione v_π legata ad una policy π che definisce la mappa (possibilmente stocastica) fra stati e azioni $A_t \sim \pi(S_t)$:

$$v_w(s) \approx v_\pi(s) = E \left[R_{t+1} + \gamma R_{t+2} + \dots | S_t = s, A_{t+i} \sim \pi(S_{t+i}), \forall i > 0 \right].$$

L'algoritmo di TD learning (Sutton, 1988) calcola l'approssimazione con l'update

$$\Delta w \propto (R_{t+1} + \gamma v_w(S_{t+1}) - v_w(S_t)) \nabla_w v_w(S_t), \quad (5.1)$$

dove $v_w(S_{t+1})$ è utilizzato per stimare il reward che si ottiene a partire dallo stato S_{t+1} . $\propto$ esprime l'uguaglianza a meno di un fattore di scala. L'equivalente della formula 5.1 per il DQN è dato dalle 4.15 e 4.16. Come descritto nel paper cui si fa riferimento, in letteratura esistono diversi esempi in cui TD learning diverge. Nello stesso paper è specificato che DQN non è soggetto a divergenza, ma l'argomentazione che segue giustifica le oscillazioni riscontrate (cfr. 5.2).

Facciamo riferimento alla figura 5.3. In questo esempio, ogni stato è descritto da una singola feature scalare ϕ tale che $\phi(s_1) = 1$ e $\phi(s_2) = 2$. Il valore stimato di ogni stato è $v(s) = w \times \phi$, dove w è il peso (parametro) che andremo ad aggiornare. In particolare, otteniamo $v(s_1) = w$ e $v(s_2) = 2w$. Queste stime sono riportate all'interno dei cerchi nella figura. Ogni reward è pari a 0, dunque le stime sarebbero perfette con $w_* = 0$.

Se aggiorniamo lo stato basandoci sulla dinamica del sistema (ossia on-policy), allora il costo dello stato s_2 verrà aggiornato più volte (in valore atteso) per ogni update del costo di s_1 .

Consideriamo adesso un update al costo dello stato s_1 ma non al costo dello stato s_2 . L'update TD di w , basandoci su 5.1, è $\Delta w \propto \gamma 2w - w = (2\gamma - 1)w$. Per un discount factor $\gamma > 1/2$, abbiamo $2\gamma > 1$ e dunque ogni peso $w \neq 0$ sarà man mano allontanato dal valore ottimale 0 e in particolare w diverge. Anche se aggiornassimo s_2 ogni volta che aggiorniamo s_1 , l'aggiornamento avverrebbe comunque meno spesso del numero di volte che avverrebbe se effettuassimo l'update on policy, cioè seguendo la dinamica del sistema, e w continuerebbe a divergere per valori di γ sufficientemente grandi.

Intuitivamente, il problema scaturito da questo esempio evidenzia che quando aggiorniamo il costo di un certo stato (per mezzo del peso w), è probabile che stiamo aggiornando anche il costo di altri stati, a causa della generalizzazione introdotta dall'approssimazione della cost function (nel caso di DQN questa è la parametrizzazione per mezzo della NN). Nel caso di learning off-policy, questo potrebbe comportare che le cost function di alcuni stati assumano valori errati.

5.1.2 Migliorare l'algoritmo DQN

Al netto di quanto discusso pocanzi, si è deciso di adottare alcuni metodi ben noti che possono arginare il problema discusso. Successivamente analizziamo i risultati prodotti.

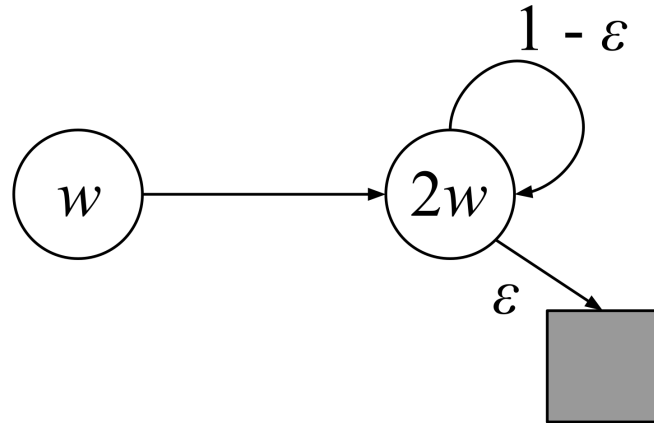


Figura 5.3. L'esempio è stato per la prima volta esposto in *An Analysis of Temporal-Difference Learning with Function Approximation* (Tsitsiklis & Van Roy, 1997). (Fonte: van Hasselt H., Hessel M., Doron Y., Sonnerat N., Strub F., Modayil J. (2018), *Deep Reinforcement Learning and the Deadly Triad*, arXiv:1812.02648.)

Experience Replay Questo metodo è trattato in *Playing Atari with Deep Reinforcement Learning*⁵, dove l'algoritmo DQN è stato per la prima volta presentato. Per comprenderne l'utilità, facciamo riferimento alla spiegazione in 5.1.1. Abbiamo visto che modificare il costo di un certo stato può "danneggiare" le stime fatte per altri stati. Una pratica che si può adottare è aggiornare più stati simultaneamente, basandoci sull'esperienza pregressa.

La tecnica di experience replay (Lin, 1992) consiste nel salvare l'esperienza dell'agente ad ogni timestep, $e_t = (s_t, a_t, r_t, s_{t+1})$ in un data-set $D = e_1, \dots, e_N$ di dimensione finita. Per una versione base è possibile utilizzare una struttura dati LIFO (viene eliminato dalla lista l'elemento che è entrato per primo, fra gli elementi presenti). Durante il loop interno dell'algoritmo, applichiamo gli update visti in 4.15, 4.16 ad un batch di dati esperienza $e \sim D$, cioè si scelgono m elementi random dal data-set. Anzichè minimizzare in base ad un singolo Q-value come in 4.13, la nuova loss function ha la forma

$$L_i(\theta_i) = \sum_{(a_t, s_t, r_t, s_{t+1}) \sim D} \left[Q^*(s_t, a_t) - Q(s_t, a_t; \theta_i) \right]^2, \quad (5.2)$$

dove Q^* è definita esattamente come nell'equazione 4.14.

Double DQN Questa miglioria proposta in nel paper *Deep Reinforcement Learning with Double Q-learning*⁶ prevede l'utilizzo di due NN, una utilizzata per approssimare la policy (policy network) che viene utilizzata nella fase on line di learning, ossia per la scelta dell'azione da intraprendere, mentre l'altra viene utilizzata come target function al momento del bootstrapping (target network). I parametri della target network vengono aggiornati mediante un soft-update utilizzando i parametri della policy network. Si ottiene che nell'equazione 4.13 la funzione Q^* è approssimata

⁵Mnih V., Kavukcuoglu K., Silver D., Graves A., Antonoglou I., Wierstra D., Riedmiller M. (2023), *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602.

⁶van Hasselt H., Guez A., Silver D. (2015), *Deep Reinforcement Learning with Double Q-learning*. arXiv:1509.06461.

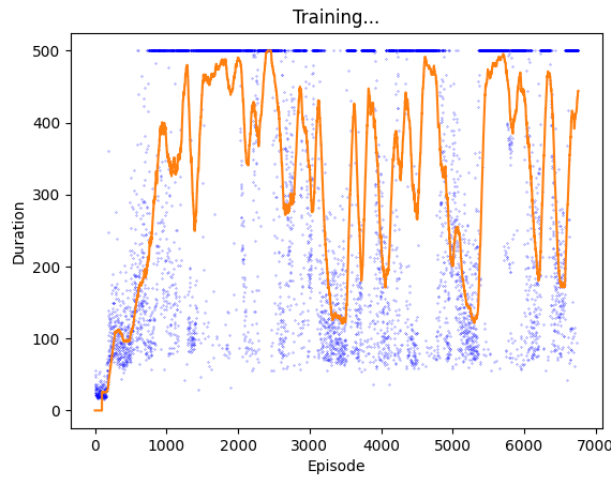


Figura 5.4. Utilizzo dell'algoritmo DDQN con Experience Replay. Il problema è analogo a quello descritto nella figura 5.2. Notiamo un netto miglioramento nella stabilità e nella stima della policy ottimale.

dalla target-network, mentre la funzione Q è approssimata dalla policy-network. Il passo di discesa del gradiente è effettuato sui parametri θ della policy-network. Ad ogni iterazione dell'algoritmo, aggiorniamo i pesi θ^* della target network mediante

$$\theta^* \leftarrow \theta^*(1 - \gamma) + \gamma\theta \quad \gamma \in (0, 1).$$

Questa tecnica riduce il fenomeno di maximization bias (Sutton & Barto, 2019) dovuto alla massimizzazione all'interno dell'update dei Q-values tramite bootstrapping. Riportiamo la spiegazione data nel libro di Barto e Sutton per il caso del Q-learning. Il libro spiega come nel Q-learning, la target policy è la policy ottenuta mediante una scelta greedy dati lo stato corrente e il guadagno delle azioni possibili. Utilizzare il guadagno massimo fra le possibili azioni come valore atteso del guadagno massimo può portare ad un significativo bias positivo. Si consideri uno stato s per cui i guadagni reali $q(s, a)$ sono tutti nulli, ma per cui i valori stimati $Q(s, a)$ sono incerti e distribuiti, alcuni hanno guadagno positivo, altri negativo. Il massimo dei valori reali è zero, ma il massimo dei valori stimati è positivo. Questo fenomeno viene chiamato maximization bias. Il problema è scaturito dal fatto che si utilizzano le coppie stato-azione sia per determinare l'azione massimizzante sia per stimare il guadagno di tale azione. Utilizzando due architetture di approssimazione possiamo scindere l'operazione di scelta del controllo da quella di stima del costo, evitando così di cadere nel meccanismo che genera il bias.

Utilizzando entrambe le migliorie appena introdotte si ottiene un algoritmo con una performance migliore rispetto a quella del DQN, in grado di offrire maggiore stabilità (riduzione delle oscillazioni nella lunghezza dell'episodio) e in generale una stima migliore della policy ottimale. I risultati sono riportati nella figura 5.4.

Multi-step bootstrapping Una componente cruciale durante il design degli algoritmi di RL è la definizione della target function (Hernandez-Garcia & Sutton, 2019). Negli algoritmi precedenti, abbiamo aggiornato i parametri della NN attraverso le minimizzazioni 4.13, chiamate da Button e Sarto bootstrapping poichè la

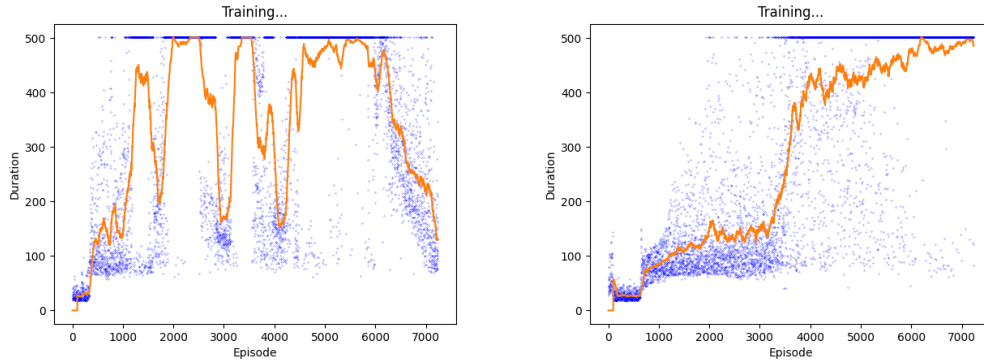


Figura 5.5. Utilizzo dell'algoritmo DDQN con Experience Replay e multistep-bootstrapping. Il problema è analogo a quello descritto nella figura 5.2. Nell'immagine a sinistra la durata degli episodi utilizzando una dimensione del batch per experience replay di 256 e eseguendo un rollout di lunghezza 2 al momento del bootstrapping. Nell'immagine a destra la durata degli episodi utilizzando una dimensione del batch per experience replay di lunghezza 512 e eseguendo un rollout di 8 al momento del bootstrapping.

target function Q^* è il valore atteso del costo della coppia stato-comando (x, u) utilizzando una policy che si assume essere quella ottimale. Una possibilità è dare l'approssimazione della target function eseguendo un rollout di un certo numero di passi, una pratica comune nel RL classico (Button & Sarto, 2019).

Definiamo il target Q-factor per coppia stato-azione (x, u) eseguendo un rollout di n step come:

$$Q_{i:i+n}^*(x_i, u_i) = \sum_{j=0}^{n-1} \gamma^j g(x_{i+j}, u_{i+j}) + \gamma^n Q(x_{i+n}, u_{i+n}; \theta^*) \quad (5.3)$$

dove le transizioni dallo stato x_{i+j} allo stato x_{i+j+1} vengono fatte utilizzando il controllo $u_{i+j} = \max_v Q(x_{i+j}, v)$. Utilizziamo DDQN, dunque due NN e θ^* rappresenta i parametri, fissati al momento del bootstrapping, della rete target. Quindi eseguiamo l'update dei parametri della NN minimizzando l'equazione

$$L_i(\theta_i) = \left[Q_{i:i+n}^*(x_i, u_i) - Q(x_i, u_i; \theta_i) \right]^2. \quad (5.4)$$

Nella figura 5.5 sono riportati i risultati ottenuti apportando tutte e tre le migliorie proposte.

5.2 Soluzione del problema CartPole ad osservabilità ridotta

Ritornando al problema ad osservabilità ridotta discusso nel secondo capitolo, ne diamo una soluzione utilizzando le NN. Si è deciso di customizzare ulteriormente l'environment, espandendo l'observation space e permettendo all'asta di puntare verso il basso. In questo modo il range di stati iniziali è ancora più ampio.

Ricordiamo che un elemento dello spazio di stato in questa versione del problema è dato da una quadrupla della forma

$$(x_i, \dot{x}_{i-1}, \theta_i, \dot{\theta}_{i-1}).$$

$\mathbf{x}(m)$	$\dot{\mathbf{x}}(m/s)$	$\theta(rad)$	$\dot{\theta}(rad/s)$	control
$[-2.4, 2.4]$	$[-\infty, +\infty]$	$[-\infty, +\infty]$	$[-\infty, +\infty]$	$\{\text{Left}, \text{Right}\}$

Tabella 5.1. Lo spazio di stato legale del nuovo environment.

Durante il testing dell'algoritmo di DQN sul problema ad osservabilità ridotta si sono riscontrati problemi di convergenza discussi nelle sezioni precedenti.

L'espansione dello spazio di stato mediante il calcolo di due nuove variabili ha portato ad una migliore performance dell'algoritmo. Si è dunque deciso di utilizzare uno spazio di stato composto da 6 variabili, fra cui l'accelerazione e l'accelerazione angolare allo step $i-2$, calcolate come $\ddot{x}_{i-2} = (\dot{x}_{i-1} - \dot{x}_{i-2})/T$ e $\ddot{\theta}_{i-2} = (\dot{\theta}_{i-1} - \dot{\theta}_{i-2})/T$. Un elemento del nuovo spazio di stato è dunque una sestupla del tipo

$$(x_i, \dot{x}_{i-1}, \ddot{x}_{i-2}, \theta_i, \dot{\theta}_{i-1}, \ddot{\theta}_{i-2}).$$

Customizzazione dell'environment Come detto, si è domificato lo spazio di stato legale dell'environment. I parametri utilizzati sono quelli della tabella 5.1.

Il reward è espresso mediante le seguenti equazioni:

$$r_\theta = \max(0, \cos \theta), \quad r_x = \left| \cos\left(\frac{x}{x_{max}} \cdot \frac{\pi}{2}\right) \right|$$

$$r = r_\theta \cdot r_x,$$

dove x_{max} è il limite superiore dell'intervallo di valori legali della variabile x , in questo caso $x_{max} := 2.4$. Il controllore viene premiato se tiene l'asta rivolta verso l'alto e il carrello in posizione centrale.

Configurazione e risultati ottenuti Si è utilizzato l'algoritmo DDQN con Experience Replay e multi-step bootstrapping. In particolare, si è utilizzata una NN con due layer nonlineare con dimensioni di input e output di 1024 e 1024. La dimensione del batch per effettuare l'experience replay utilizzata è di 512 elementi, con dimensione della memoria D in cui è stata salvata l'esperienza passata, e dalla quale si è effettuato il sampling delle batches, di 20000 elementi. Si sono utilizzati 8 step di rollout al momento del bootstrapping. L'aggiornamento dei pesi della target network avviene con una costante di aggiornamento di 0.02, al termine di ogni episodio.

Si sono salvati i parametri con cui si sono ottenuti i risultati migliori, ovvero quando il reward medio degli ultimi 100 episodi era di 415 steps (un ottimo risultato considerando che la posizione iniziale potrebbe l'asta che punta verso il basso). Il risultato può essere visionato su: <https://youtu.be/R1rWI6UsQYQ>.

5.3 Utilizzo di DQN per altri problemi di controllo e analisi dei risultati

Abbiamo impiegato il DQN per risolvere altri i problemi proposti nella libreria gymnasium. In particolare, si sono implementati gli environment *MountainCar* (5.6 e 5.7), *Acrobot* (5.8 e 5.9) e *LunarLander* (5.13 e 5.13).

MountainCar Citando Gymnasium (<https://gymnasium.farama.org/>), "Il Mountain Car Markov Decision Process (MDP) è un MDP deterministico che consiste in un'auto posizionata stocasticamente sul fondo di una valle sinusoidale, con le uniche azioni possibili che sono le accelerazioni che possono essere applicate all'auto in entrambe le direzioni. L'obiettivo dell'MDP è accelerare strategicamente l'auto per raggiungere lo stato di destinazione in cima alla collina destra. Esistono due versioni del dominio della montagna in palestra: una con azioni discrete e una con continue. Questa versione è quella con azioni discrete.

Questo MDP è apparso per la prima volta nella tesi di dottorato di Andrew Moore (1990), *Efficient memory-based learning for robot control*."

Lo spazio di stato è composto da due variabili: la posizione della macchinina lungo l'asse delle ascisse e la velocità della macchinina. Lo spazio delle azioni è discreto con 3 possibili stati: *accelerazione verso sinistra*, *nessuna accelerazione*, *accelerazione verso destra*. Per ulteriori dettagli si rimanda il lettore a https://gymnasium.farama.org/environments/classic_control/mountain_car/.

Si è utilizzato un reward sistem personalizzato, prendendo spunto da <https://github.com/nitish-kalan/MountainCar-v0-Deep-Q-Learning-DQN-Keras>. Si assegna un reward di 250 se la macchinina raggiunge l'obiettivo, altrimenti il reward è proporzionale alla distanza percorsa dalla macchinina rispetto alla sua posizione precedente. In questo modo si velocizzano i tempi di training. Nella figura 5.7 sono riportati i risultati del training con 1000 episodi utilizzando 3 versioni diverse dell'algoritmo.

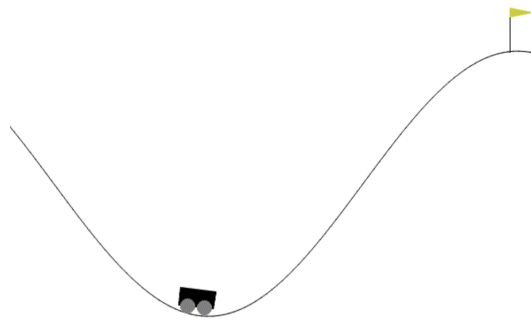


Figura 5.6. L'environment MountainCar di Gymnasium. L'obiettivo è portare la macchinina alla bandiera nel minor numero di step.

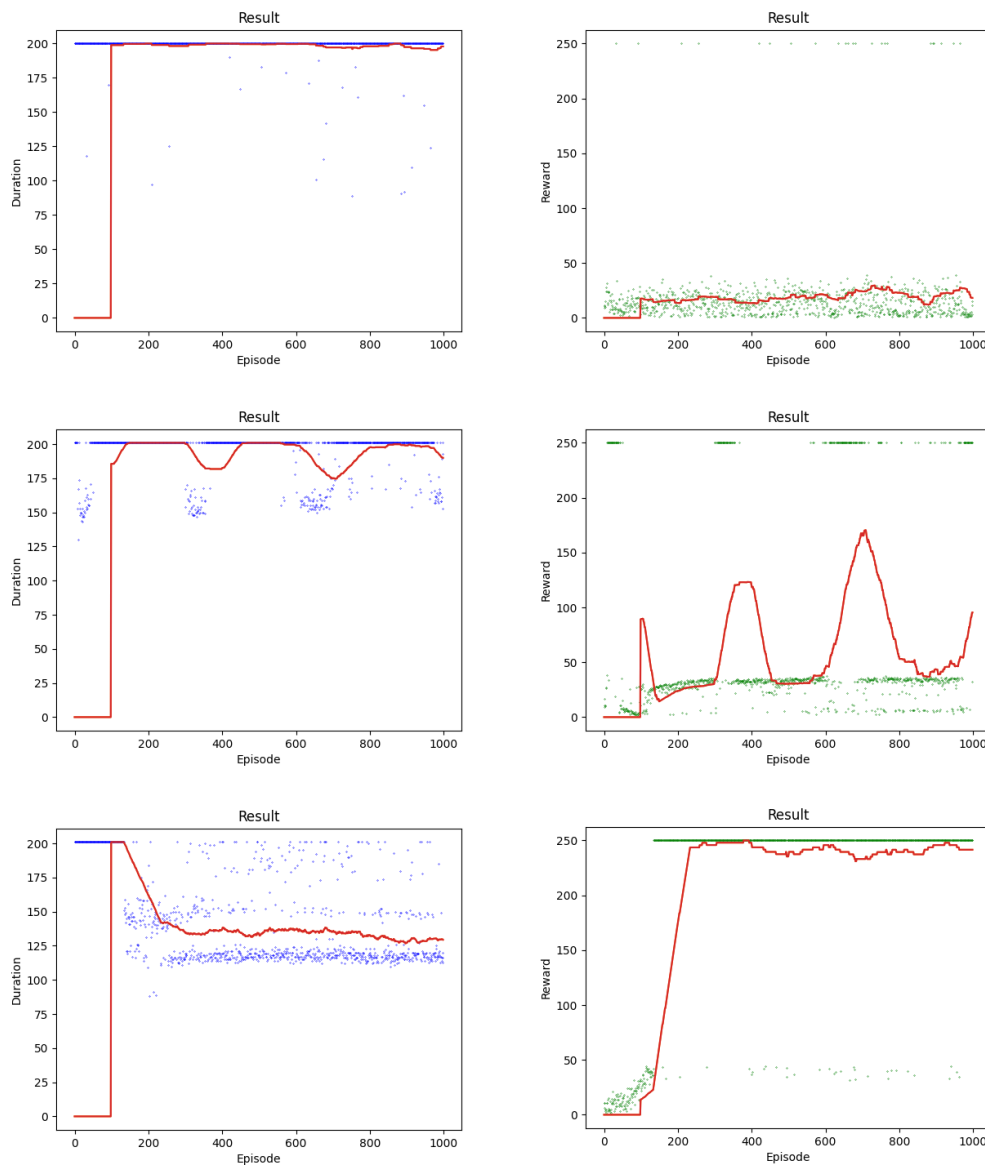


Figura 5.7. A sinistra i grafici del reward ottenuto ad ogni episodio, a destra la durata degli episodi, in rosso la durata media. Gli algoritmi utilizzati sono (dall'alto verso il basso) DQN standard, Double DQN con Experience Replay, Double DQN con Experience Replay e multi-step bootstrapping (8 step). DQN standard non riesce a risolvere il problema. DDQN con E.R. ottiene risultati oscillanti, con brevi periodi in cui vi è convergenza ai Q-factors ottimali. DDQN con E.R e multi-step bootstrapping ha una pseudo-convergenza ai Q-factors ottimali.

Acrobot Citando Gymnasium (<https://gymnasium.farama.org/>), "L'ambiente Acrobot si basa sul lavoro di Sutton in *Generalization in Reinforcement Learning: Successful Samples Using Sparse Coarse Coding* (1995) e sul libro di Sutton e Barto *Reinforcement Learning, An Introduction, Second Edition*. Il sistema è costituito da due maglie collegate linearmente per formare una catena, con un'estremità fissa. Viene azionato il giunto tra i due collegamenti. L'obiettivo è applicare una forza di torsione sul giunto azionato per far oscillare l'estremità libera della catena lineare

al di sopra di una determinata altezza partendo dallo stato iniziale di sospensione verso il basso".

Lo spazio di stato dell'environment Acrobot è continuo di dimensione 6. In particolare sono considerati i seni e coseni degli angoli formati dalle aste attaccate ai perni (cfr. figura 5.8) e le velocità angolari. Per ulteriori dettagli si rimanda a https://gymnasium.farama.org/environments/classic_control/acrobot/.

Lo spazio dei controlli è discreto di dimensione 1 con 3 possibili stati: applicazione di forza nulla, -1 e 1 alla giuntura azionata.

Ogni istante in cui il braccio è al di sotto della linea nera ha un costo di -1 .

Nella figura 5.9 sono riportati i risultati di 1000 episodi di esecuzione utilizzando l'algoritmo DQN nella sua versione standard, Double DQN con Experience Replay, Double DQN con Experience Replay e multi-step (8) bootstrapping.

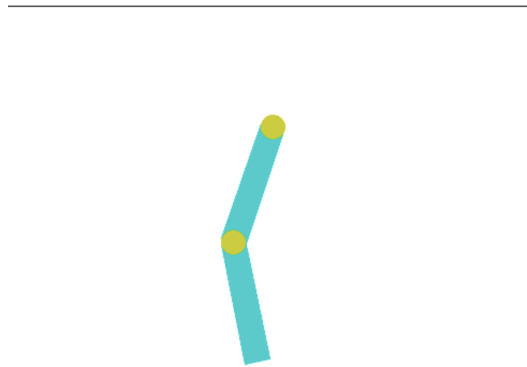


Figura 5.8. L'environment Acrobot di Gymnasium (<https://gymnasium.farama.org/>). L'obiettivo è portare il doppio pendolo al di là della linea nera, nel minor tempo possibile.

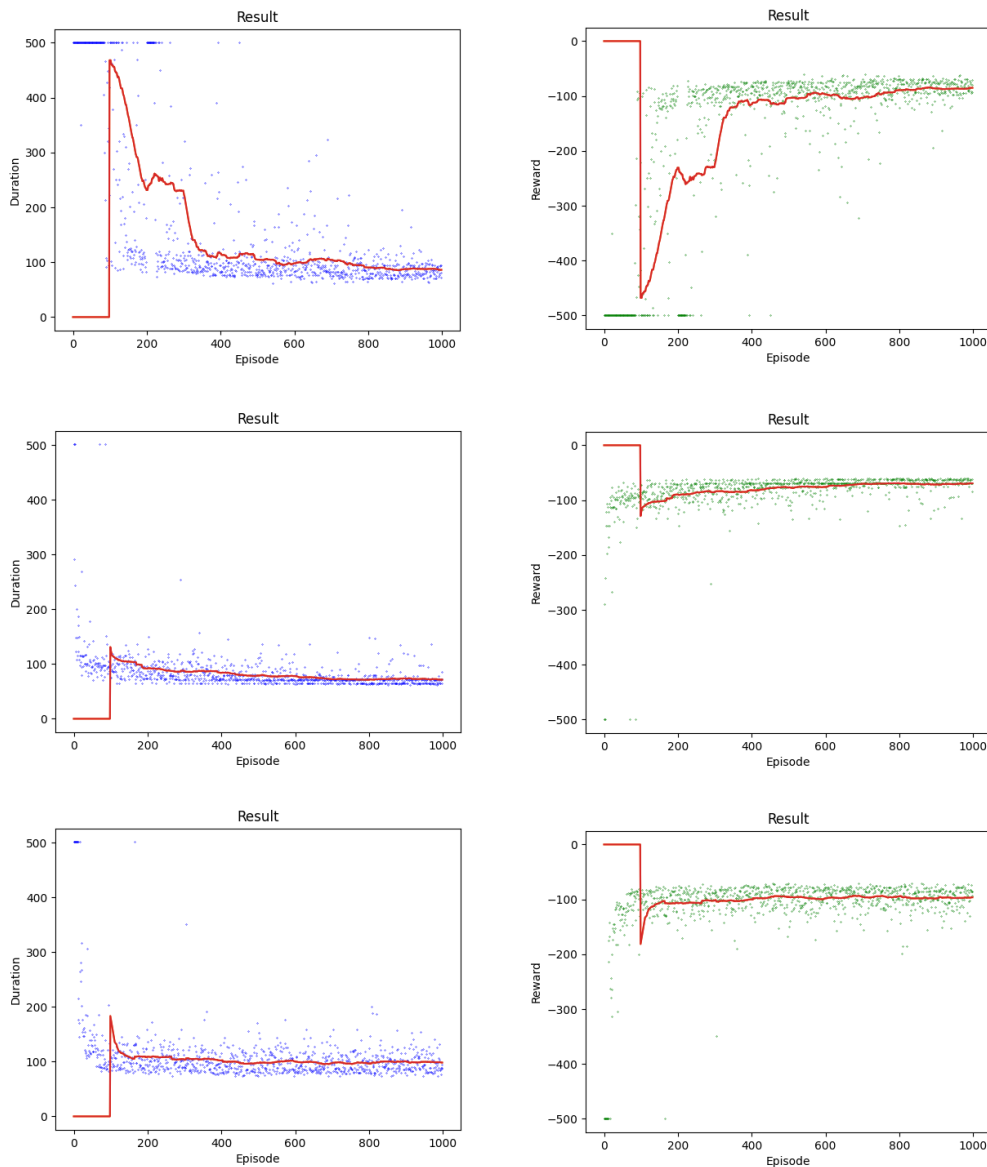


Figura 5.9. A sinistra i grafici del reward ottenuto ad ogni episodio, a destra la durata degli episodi, in rosso la durata media. Gli algoritmi utilizzati sono (dall'alto verso il basso) DQN standrad, Double DQN con Experience Replay, Double DQN con Experience Replay e multi-step bootstrapping (8 step). In questo caso, tutti gli algoritmi danno una soluzione al problema. Si ha convergenza ai Q-factors ottimali.

Oltre alla versione standard dell'Acrobot, si è testato l'algoritmo anche su una versione modificata in cui l'obiettivo è mantenere l'asta in equilibrio (figura 5.10). Riportiamo nella figura 5.11 il risultato del training impiegando DDQN con Experience Replay. La difficoltà principale è stata trovare un corretto sistema di reward ed adeguati parametri dell'algoritmo per non rimanere in minimi locali. Si assegna una penalità di -1 ogni volta che l'asta si trova in posizione non verticale, per il sistema di reward si è preso spunto da <https://github.com/A-make/gym-custom>.

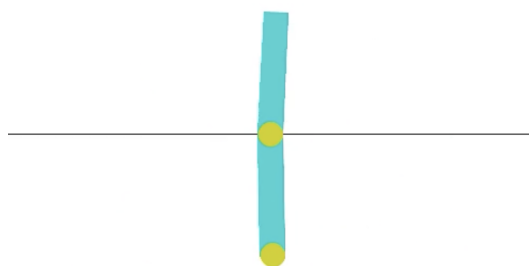


Figura 5.10. Acrobot con modifica. L'obiettivo è mantenere il doppio pendolo in equilibrio.

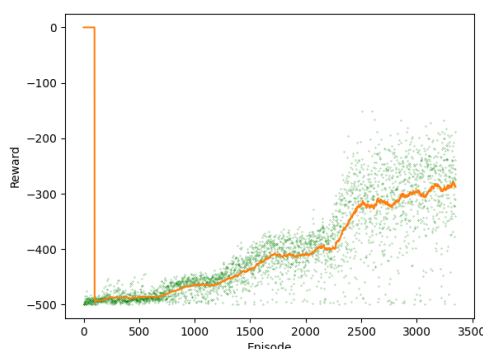


Figura 5.11. I risultati del training per Acrobot customizzato. Il controllore mantiene l'asta in equilibrio in circa 250/500 step. Va considerato il periodo necessario a portare il doppio pendolo in posizione verticale, di circa 250 step.

LunarLander Lo spazio di stato dell'environment LunarLander è composto da 8 variabili continue: le coordinate x, y della navicella, le velocità direzionali rispetto ad x e y , la velocità angolare della navicella e due variabili booleane che descrivono se ciascun piede della navicella sta toccando terra o meno. Per ulteriori dettagli si rimanda il lettore a https://gymnasium.farama.org/environments/box2d/lunar_lander/.

Lo spazio dei controlli è discreto di dimensione 1 con 4 possibili stati: 0, 1, 2, 3, corrispondenti rispettivamente ai controlli *non far nulla*, *utilizza il motore sinistro*, *utilizza il motore principale*, *utilizza il motore destro*.

Per il reward system si rimanda il lettore al sito sopra citato, mentre gli stati terminanti sono raggiunti nei casi in cui la navicella esce dallo spazio visibile, tocca terra con il corpo della navicella, resta in uno stato di immobilità all'infuori dell'area demarcata dalle due bandierine.

Nella figura 5.13 sono riportati i risultati di 2000 episodi di esecuzione utilizzando l'algoritmo DQN nella sua versione standard, Double DQN con Experience Replay, Double DQN con Experience Replay e multi-step (8) bootstrapping.

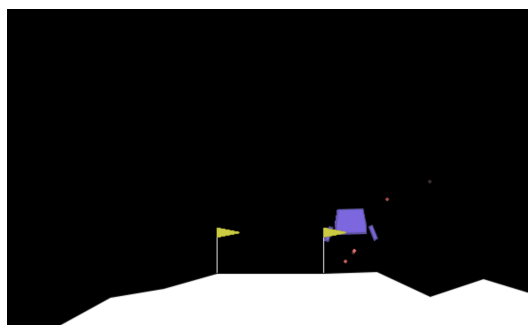


Figura 5.12. L'environnement LunarLander di Gymnasium. L'obiettivo è far atterrare la navicella fra le due bandierine.

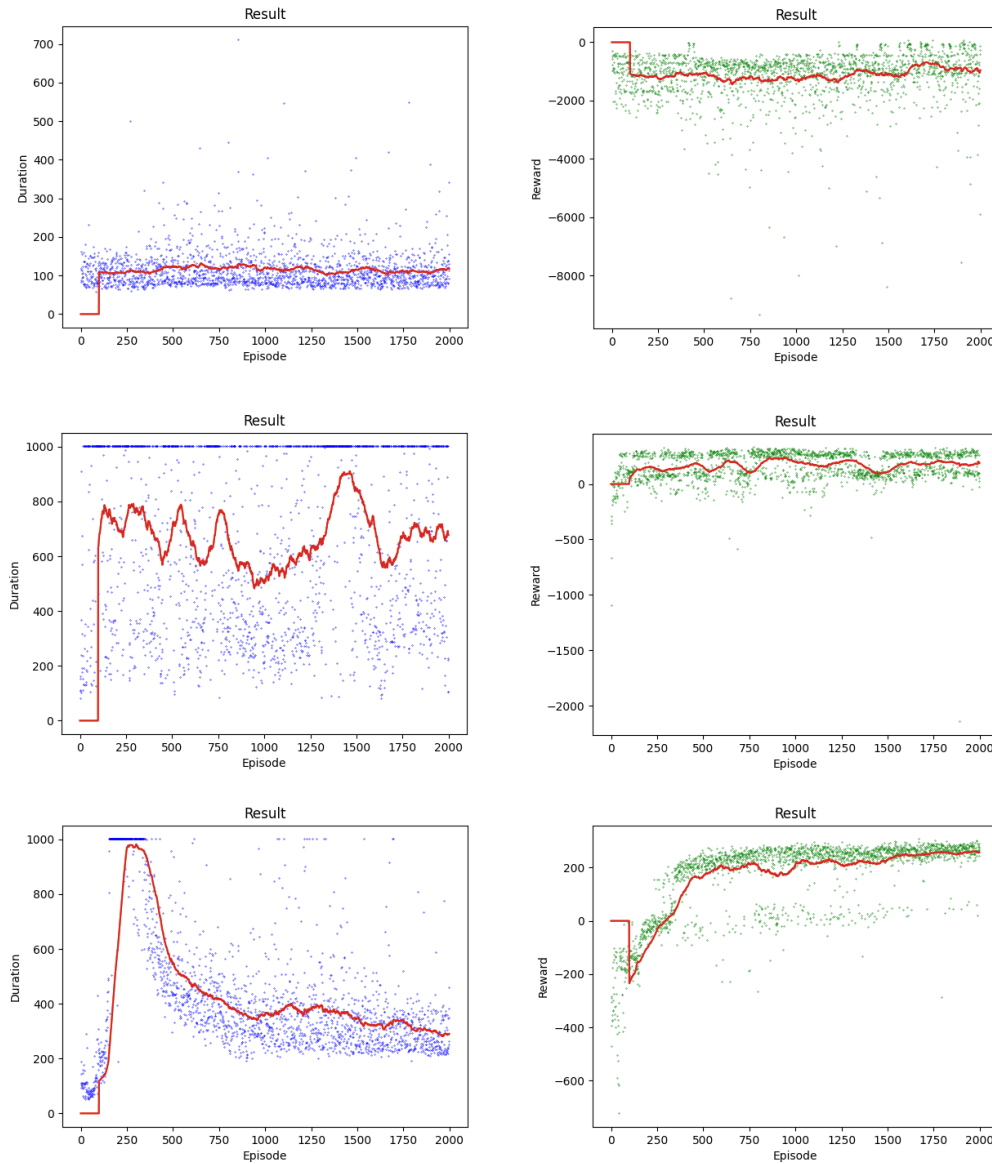


Figura 5.13. A sinistra i grafici del reward ottenuto ad ogni episodio, a destra la durata degli episodi, in rosso la durata media. Gli algoritmi utilizzati sono (dall'alto verso il basso) DQN standrad, Double DQN con Experience Replay, Double DQN con Experience Replay e multi-step bootstrapping (8 step).

In questo caso, i tre algoritmi convergono a dei minimi locali (è possibile ottenere reward > 200). DDQN con E.R. e multi-step bootstrapping perfoma meglio degli altri anche in questo caso. Con un numero maggiore di episodi e aggiustando il parametro di action selection è possibile raggiungere la convergenza verso i Q-factors ottimali.

Analizzando i risultati ottenuti nelle figure 5.7, 5.9, 5.13 e i risultati ottenuti precedentemente per il problema CartPole (figure 5.2, 5.4, 5.5), concludiamo che le migliorie introdotte irrobustiscono notevolmente le performance dell'algoritmo DQN. In particolare, la versione con multi-step bootstrapping garantisce in tutti i casi posti sotto esame una pseudo convergenza alla cost-function ottimale, priva di oscillazioni nel rendimento.

Capitolo 6

Conclusioni

Dopo aver esaminato i risultati ottenuti, possiamo trarre le nostre conclusioni individuando i passaggi più significativi.

La soluzione con la Dynamic Programming assicurava una convergenza alla cost-function ottimale; ci siamo però dovuti scontrare con i problemi legati al suo costo computazionale.

L'utilizzo delle Neural Networks ci ha permesso di affrontare problemi con spazio di stato continuo ed in tempi ragionevoli; le performance dell'algoritmo erano però a questo stadio troppo poco stabili per considerare risolti i problemi proposti: l'algoritmo applicato al problema CartPole con variabili oscure risultava molto instabile anche nelle versioni con spazio di stato meno "impegnativo".

Tramite le migliorie discusse nel capitolo 5 siamo finalmente riusciti a produrre risultati soddisfacenti. In particolare, arricchendo l'algoritmo DQN proposto in *Playing Atari with Deep Reinforcement Learning*¹, con il multi-step bootstrapping² e il Double DQN³, abbiamo ottenuto una pseudo-convergenza alla cost-function ottimale in ognuna delle applicazioni esaminate.

¹Mnih V., Kavukcuoglu K., Silver D., Graves A., Antonoglou I., Wierstra D., Riedmiller M. (2023), *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602.

²Hernandez-Garcia J.F., Sutton R.S. (2019), *Understanding Multi-Step Deep Reinforcement Learning: A Systematic Study of the DQN Target*, arXiv:1901.07510.

³van Hasselt H., Guez A., Silver D. (2015), *Deep Reinforcement Learning with Double Q-learning*. arXiv:1509.06461.

Bibliografia

- [1] Bertsekas D.P. (2019), *Reinforcement Learning and Optimal Control*. Belmont, Massachusetts, Athena Scientific.
- [2] Bertsekas D.P. (2023), *A Course in Reinforcement Learning*. Belmont, Massachusetts, Athena Scientific.
- [3] Sutton R.S., Barto A.G. (2018), *Reinforcement Learning, An Introduction, Second Edition*. Cambridge, Massachusetts, The MIT Press.
- [4] Mnih V., Kavukcuoglu K., Silver D., Rusu A.A., Veness J., Bellemare M.G., Graves A., Riedmiller M., Fidjeland A.K., Ostrovski G., Petersen S., Beattie C., Sadik A., Antonoglou I., King H., Kumaran D., Wierstra D., Legg S., Hassabis D. (2015), *Human-level control through deep reinforcement learning*. Nature, 518:529-533.
- [5] Mnih V., Kavukcuoglu K., Silver D., Graves A., Antonoglou I., Wierstra D., Riedmiller M. (2023), *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602.
- [6] van Hasselt H., Guez A., Silver D. (2015), *Deep Reinforcement Learning with Double Q-learning*. arXiv:1509.06461.
- [7] van Hasselt H., Hessel M., Doron Y., Sonnerat N., Strub F., Modayil J. (2018), *Deep Reinforcement Learning and the Deadly Triad*. arXiv:1812.02648.
- [8] Paszke A., Towers M. (2024), *Pytorch Reinforcement Learning (DQN) Tutorial*. https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html.
- [9] Hernandez-Garcia J.F., Sutton R.S. (2019), *Understanding Multi-Step Deep Reinforcement Learning: A Systematic Study of the DQN Target*. arXiv:1901.07510.
- [10] Kumar S. (2019), *Balancing a CartPole System with Reinforcement Learning - A Tutorial*. arXiv:2006.04938.
- [11] Watkins C.J.C.H. (1992), *Technical Note, Q-learning*. Machine Learning, 8:279-292.
- [12] Lin L. (1992), *Self-Improving Reactive Agents Based On Reinforcement Learning, Planning and Teaching*. Machine Learning, 8:293-321.
- [13] Hastie T., Tibshirani R., Friedman J. (2009), *The Elements of Statistical Learning. Data Mining, Inference, and Prediction, Second Edition*. New York, New York, Springer.